13. Conversions

A ***conversion*** enables an expression of one type to be treated as another type. Conversions can be ***implicit*** or ***explicit***, and this determines whether an explicit cast is required. [*Example:* For instance, the conversion from type `int` to type `long` is implicit, so expressions of type `int` can implicitly be treated as type `long`. The opposite conversion, from type `long` to type `int`, is explicit and so an explicit cast is required.

```
int a = 123;
long b = a;       // implicit conversion from int to long
int c = (int) b;  // explicit conversion from long to int
```

end example] Some conversions are defined by the language. Programs may also define their own conversions (§13.4).

13.1 Implicit conversions

The following conversions are classified as implicit conversions:

- Identity conversions
- Implicit numeric conversions
- Implicit enumeration conversions.
- Implicit reference conversions
- Boxing conversions
- Implicit constant expression conversions
- User-defined implicit conversions

Implicit conversions can occur in a variety of situations, including function member invocations (§14.4.3), cast expressions (§14.6.6), and assignments (§14.13).

The pre-defined implicit conversions always succeed and never cause exceptions to be thrown. [*Note:* Properly designed user-defined implicit conversions should exhibit these characteristics as well. *end note*]

13.1.1 Identity conversion

An identity conversion converts from any type to the same type. This conversion exists only such that an entity that already has a required type can be said to be convertible to that type.

13.1.2 Implicit numeric conversions

The implicit numeric conversions are:

- From `sbyte` to `short`, `int`, `long`, `float`, `double`, or `decimal`.
- From `byte` to `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, or `decimal`.
- From `short` to `int`, `long`, `float`, `double`, or `decimal`.
- From `ushort` to `int`, `uint`, `long`, `ulong`, `float`, `double`, or `decimal`.
- From `int` to `long`, `float`, `double`, or `decimal`.
- From `uint` to `long`, `ulong`, `float`, `double`, or `decimal`.
- From `long` to `float`, `double`, or `decimal`.
- From `ulong` to `float`, `double`, or `decimal`.
- From `char` to `ushort`, `int`, `uint`, `long`, `ulong`, `float`, `double`, or `decimal`.
- From `float` to `double`.

Conversions from `int`, `uint`, or `long` to `float` and from `long` to `double` may cause a loss of precision, but will never cause a loss of magnitude. The other implicit numeric conversions never lose any information.

There are no implicit conversions to the `char` type, so values of the other integral types do not automatically convert to the `char` type.

13.1.3 Implicit enumeration conversions

An implicit enumeration conversion permits the *decimal-integer-literal* `0` to be converted to any *enum-type*.

13.1.4 Implicit reference conversions

The implicit reference conversions are:

- From any *reference-type* to `object`.
- From any *class-type* `S` to any *class-type* `T`, provided `S` is derived from `T`.
- From any *class-type* `S` to any *interface-type* `T`, provided `S` implements `T`.
- From any *interface-type* `S` to any *interface-type* `T`, provided `S` is derived from `T`.
- From an *array-type* `S` with an element type S_E to an *array-type* `T` with an element type T_E, provided all of the following are true:
  - `S` and `T` differ only in element type. In other words, `S` and `T` have the same number of dimensions.
  - Both S_E and T_E are *reference-type*s.
  - An implicit reference conversion exists from S_E to T_E.
- From any *array-type* to `System.Array`.
- From any *delegate-type* to `System.Delegate`.
- From any *array-type* or *delegate-type* to `System.ICloneable`.
- From the null type to any *reference-type*.

The implicit reference conversions are those conversions between *reference-type*s that can be proven to always succeed, and therefore require no checks at run-time.

Reference conversions, implicit or explicit, never change the referential identity of the object being converted. [*Note:* In other words, while a reference conversion may change the type of the reference, it never changes the type or value of the object being referred to. *end note*]

13.1.5 Boxing conversions

A boxing conversion permits any *value-type* to be implicitly converted to the type `object` or to any *interface-type* implemented by the *value-type*. Boxing a value of a *value-type* consists of allocating an object instance and copying the *value-type* value into that instance.

Boxing conversions are described further in §11.3.1.

13.1.6 Implicit constant expression conversions

An implicit constant expression conversion permits the following conversions:

- A *constant-expression* (§14.15) of type `int` can be converted to type `sbyte`, `byte`, `short`, `ushort`, `uint`, or `ulong`, provided the value of the *constant-expression* is within the range of the destination type.
- A *constant-expression* of type `long` can be converted to type `ulong`, provided the value of the *constant-expression* is not negative.

13.1.7 User-defined implicit conversions

A user-defined implicit conversion consists of an optional standard implicit conversion, followed by execution of a user-defined implicit conversion operator, followed by another optional standard implicit conversion. The exact rules for evaluating user-defined conversions are described in §13.4.3.

13.2 Explicit conversions

The following conversions are classified as explicit conversions:

- All implicit conversions.
- Explicit numeric conversions.
- Explicit enumeration conversions.
- Explicit reference conversions.
- Explicit interface conversions.
- Unboxing conversions.
- User-defined explicit conversions.

Explicit conversions can occur in cast expressions (§14.6.6).

The set of explicit conversions includes all implicit conversions. [*Note:* This means that redundant cast expressions are allowed. *end note*]

The explicit conversions that are not implicit conversions are conversions that cannot be proven to always succeed, conversions that are known to possibly lose information, and conversions across domains of types sufficiently different to merit explicit notation.

13.2.1 Explicit numeric conversions

The explicit numeric conversions are the conversions from a *numeric-type* to another *numeric-type* for which an implicit numeric conversion (§13.1.2) does not already exist:

- From sbyte to byte, ushort, uint, ulong, or char.
- From byte to sbyte and char.
- From short to sbyte, byte, ushort, uint, ulong, or char.
- From ushort to sbyte, byte, short, or char.
- From int to sbyte, byte, short, ushort, uint, ulong, or char.
- From uint to sbyte, byte, short, ushort, int, or char.
- From long to sbyte, byte, short, ushort, int, uint, ulong, or char.
- From ulong to sbyte, byte, short, ushort, int, uint, long, or char.
- From char to sbyte, byte, or short.
- From float to sbyte, byte, short, ushort, int, uint, long, ulong, char, or decimal.
- From double to sbyte, byte, short, ushort, int, uint, long, ulong, char, float, or decimal.
- From decimal to sbyte, byte, short, ushort, int, uint, long, ulong, char, float, or double.

Because the explicit conversions include all implicit and explicit numeric conversions, it is always possible to convert from any *numeric-type* to any other *numeric-type* using a cast expression (§14.6.6).

The explicit numeric conversions possibly lose information or possibly cause exceptions to be thrown. An explicit numeric conversion is processed as follows:

- For a conversion from an integral type to another integral type, the processing depends on the overflow checking context (§14.5.12) in which the conversion takes place:
  - In a checked context, the conversion succeeds if the value of the source operand is within the range of the destination type, but throws a System.OverflowException if the value of the source operand is outside the range of the destination type.
  - In an unchecked context, the conversion always succeeds, and proceeds as follows.
    - If the source type is larger than the destination type, then the source value is truncated by discarding its “extra” most significant bits. The result is then treated as a value of the destination type.
    - If the source type is smaller than the destination type, then the source value is either sign-extended or zero-extended so that it is the same size as the destination type. Sign-extension is used if the source type is signed; zero-extension is used if the source type is unsigned. The result is then treated as a value of the destination type.
    - If the source type is the same size as the destination type, then the source value is treated as a value of the destination type
- For a conversion from decimal to an integral type, the source value is rounded towards zero to the nearest integral value, and this integral value becomes the result of the conversion. If the resulting integral value is outside the range of the destination type, a System.OverflowException is thrown.
- For a conversion from floator double to an integral type, the processing depends on the overflow-checking context (§14.5.12) in which the conversion takes place:
  - In a checked context, the conversion proceeds as follows:
    - If the value of the source operand is within the range of the destination type, then it is rounded towards zero to the nearest integral value of the destination type, and this integral value is the result of the conversion.
    - Otherwise, a System.OverflowException is thrown.

- In an `unchecked` context, the conversion always succeeds, and proceeds as follows.
  - If the value of the source operand is within the range of the destination type, then it is rounded towards zero to the nearest integral value of the destination type, and this integral value is the result of the conversion.
  - Otherwise, the result of the conversion is an unspecified value of the destination type.

- For a conversion from `double` to `float`, the `double` value is rounded to the nearest `float` value. If the `double` value is too small to represent as a `float`, the result becomes positive zero or negative zero. If the `double` value is too large to represent as a `float`, the result becomes positive infinity or negative infinity. If the `double` value is NaN, the result is also NaN.

- For a conversion from `float` or `double` to `decimal`, the source value is converted to `decimal` representation and rounded to the nearest number after the 28th decimal place if required (§11.1.6). If the source value is too small to represent as a `decimal`, the result becomes zero. If the source value is NaN, infinity, or too large to represent as a `decimal`, a `System.OverflowException` is thrown.

- For a conversion from `decimal` to `float` or `double`, the `decimal` value is rounded to the nearest `double` or `float` value. While this conversion may lose precision, it never causes an exception to be thrown.

13.2.2 Explicit enumeration conversions

The explicit enumeration conversions are:

- From `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, or `decimal` to any *enum-type*.

- From any *enum-type* to `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, or `decimal`.

- From any *enum-type* to any other *enum-type*.

An explicit enumeration conversion between two types is processed by treating any participating *enum-type* as the underlying type of that *enum-type*, and then performing an implicit or explicit numeric conversion between the resulting types. For example, given an *enum-type* `E` with and underlying type of `int`, a conversion from `E` to `byte` is processed as an explicit numeric conversion (§13.2.1) from `int` to `byte`, and a conversion from `byte` to `E` is processed as an implicit numeric conversion (§13.1.2) from `byte` to `int`.

13.2.3 Explicit reference conversions

The explicit reference conversions are:

- From `object` to any *reference-type*.

- From any *class-type* `S` to any *class-type* `T`, provided `S` is a base class of `T`.

- From any *class-type* `S` to any *interface-type* `T`, provided `S` is not sealed and provided `S` does not implement `T`.

- From any *interface-type* `S` to any *class-type* `T`, provided `T` is not sealed or provided `T` implements `S`.

- From any *interface-type* `S` to any *interface-type* `T`, provided `S` is not derived from `T`.

- From an *array-type* `S` with an element type S_E to an *array-type* `T` with an element type T_E, provided all of the following are true:
  - `S` and `T` differ only in element type. (In other words, `S` and `T` have the same number of dimensions.)
  - Both S_E and T_E are *reference-type*s.
  - An explicit reference conversion exists from S_E to T_E.

- From `System.Array` and the interfaces it implements, to any *array-type*.
- From `System.Delegate` and the interfaces it implements, to any *delegate-type*.

The explicit reference conversions are those conversions between reference-types that require run-time checks to ensure they are correct.

For an explicit reference conversion to succeed at run-time, the value of the source operand must be `null`, or the *actual* type of the object referenced by the source operand must be a type that can be converted to the destination type by an implicit reference conversion (§13.1.4). If an explicit reference conversion fails, a `System.InvalidCastException` is thrown.

Reference conversions, implicit or explicit, never change the referential identity of the object being converted. [*Note:* In other words, while a reference conversion may change the type of the reference, it never changes the type or value of the object being referred to. *end note*]

13.2.4 Unboxing conversions

An unboxing conversion permits an explicit conversion from type `object` to any *value-type* or from any *interface-type* to any *value-type* that implements the *interface-type*. An unboxing operation consists of first checking that the object instance is a boxed value of the given *value-type*, and then copying the value out of the instance.

Unboxing conversions are described further in §11.3.2.

13.2.5 User-defined explicit conversions

A user-defined explicit conversion consists of an optional standard explicit conversion, followed by execution of a user-defined implicit or explicit conversion operator, followed by another optional standard explicit conversion. The exact rules for evaluating user-defined conversions are described in §13.4.4.

13.3 Standard conversions

The standard conversions are those pre-defined conversions that can occur as part of a user-defined conversion.

13.3.1 Standard implicit conversions

The following implicit conversions are classified as standard implicit conversions:

- Identity conversions (§13.1.1)
- Implicit numeric conversions (§13.1.2)
- Implicit reference conversions (§13.1.4)
- Boxing conversions (§13.1.5)
- Implicit constant expression conversions (§13.1.6)

The standard implicit conversions specifically exclude user-defined implicit conversions.

13.3.2 Standard explicit conversions

The standard explicit conversions are all standard implicit conversions plus the subset of the explicit conversions for which an opposite standard implicit conversion exists. [*Note:* In other words, if a standard implicit conversion exists from a type `A` to a type `B`, then a standard explicit conversion exists from type `A` to type `B` and from type `B` to type `A`. *end note*]

13.4 User-defined conversions

C# allows the pre-defined implicit and explicit conversions to be augmented by ***user-defined conversions***. User-defined conversions are introduced by declaring conversion operators (§17.9.3) in class and struct types.

13.4.1 Permitted user-defined conversions

C# permits only certain user-defined conversions to be declared. In particular, it is not possible to redefine an already existing implicit or explicit conversion. A class or struct is permitted to declare a conversion from a source type `S` to a target type `T` only if all of the following are true:

- `S` and `T` are different types.
- Either `S` or `T` is the class or struct type in which the operator declaration takes place.
- Neither `S` nor `T` is `object` or an *interface-type*.
- `T` is not a base class of `S`, and `S` is not a base class of `T`.

The restrictions that apply to user-defined conversions are discussed further in §17.9.3.

13.4.2 Evaluation of user-defined conversions

A user-defined conversion converts a value from its type, called the ***source type***, to another type, called the ***target type***. Evaluation of a user-defined conversion centers on finding the ***most specific*** user-defined conversion operator for the particular source and target types. This determination is broken into several steps:

- Finding the set of classes and structs from which user-defined conversion operators will be considered. This set consists of the source type and its base classes and the target type and its base classes (with the implicit assumptions that only classes and structs can declare user-defined operators, and that non-class types have no base classes).
- From that set of types, determining which user-defined conversion operators are applicable. For a conversion operator to be applicable, it must be possible to perform a standard conversion (§13.3) from the source type to the operand type of the operator, and it must be possible to perform a standard conversion from the result type of the operator to the target type.
- From the set of applicable user-defined operators, determining which operator is unambiguously the most specific. In general terms, the most specific operator is the operator whose operand type is “closest” to the source type and whose result type is “closest” to the target type. The exact rules for establishing the most specific user-defined conversion operator are defined in the following sections.

Once a most specific user-defined conversion operator has been identified, the actual execution of the user-defined conversion involves up to three steps:

- First, if required, performing a standard conversion from the source type to the operand type of the user-defined conversion operator.
- Next, invoking the user-defined conversion operator to perform the conversion.
- Finally, if required, performing a standard conversion from the result type of the user-defined conversion operator to the target type.

Evaluation of a user-defined conversion never involves more than one user-defined conversion operator. In other words, a conversion from type `S` to type `T` will never first execute a user-defined conversion from `S` to `X` and then execute a user-defined conversion from `X` to `T`.

Exact definitions of evaluation of user-defined implicit or explicit conversions are given in the following sections. The definitions make use of the following terms:

- If a standard implicit conversion (§13.3.1) exists from a type A to a type B, and if neither A nor B are *interface-type*s, then A is said to be ***encompassed by*** B, and B is said to ***encompass*** A.

- The ***most encompassing type*** in a set of types is the one type that encompasses all other types in the set. If no single type encompasses all other types, then the set has no most encompassing type. In more intuitive terms, the most encompassing type is the "largest" type in the set—the one type to which each of the other types can be implicitly converted.

- The ***most encompassed type*** in a set of types is the one type that is encompassed by all other types in the set. If no single type is encompassed by all other types, then the set has no most encompassed type. In more intuitive terms, the most encompassed type is the "smallest" type in the set—the one type that can be implicitly converted to each of the other types.

13.4.3 User-defined implicit conversions

A user-defined implicit conversion from type S to type T is processed as follows:

- Find the set of types, D, from which user-defined conversion operators will be considered. This set consists of S (if S is a class or struct), the base classes of S (if S is a class), T (if T is a class or struct), and the base classes of T (if T is a class).

- Find the set of applicable user-defined conversion operators, U. This set consists of the user-defined implicit conversion operators declared by the classes or structs in D that convert from a type encompassing S to a type encompassed by T. If U is empty, the conversion is undefined and a compile-time error occurs.

- Find the most specific source type, S_X, of the operators in U:
  - If any of the operators in U convert from S, then S_X is S.
  - Otherwise, S_X is the most encompassed type in the combined set of source types of the operators in U. If no most encompassed type can be found, then the conversion is ambiguous and a compile-time error occurs.

- Find the most specific target type, T_X, of the operators in U:
  - If any of the operators in U convert to T, then T_X is T.
  - Otherwise, T_X is the most encompassing type in the combined set of target types of the operators in U. If no most encompassing type can be found, then the conversion is ambiguous and a compile-time error occurs.

- If U contains exactly one user-defined conversion operator that converts from S_X to T_X, then this is the most specific conversion operator. If no such operator exists, or if more than one such operator exists, then the conversion is ambiguous and a compile-time error occurs. Otherwise, the user-defined conversion is applied:
  - If S is not S_X, then a standard implicit conversion from S to S_X is performed.
  - The most specific user-defined conversion operator is invoked to convert from S_X to T_X.
  - If T_X is not T, then a standard implicit conversion from T_X to T is performed.

13.4.4 User-defined explicit conversions

A user-defined explicit conversion from type S to type T is processed as follows:

- Find the set of types, D, from which user-defined conversion operators will be considered. This set consists of S (if S is a class or struct), the base classes of S (if S is a class), T (if T is a class or struct), and the base classes of T (if T is a class).
- Find the set of applicable user-defined conversion operators, U. This set consists of the user-defined implicit or explicit conversion operators declared by the classes or structs in D that convert from a type encompassing or encompassed by S to a type encompassing or encompassed by T. If U is empty, the conversion is undefined and a compile-time error occurs.
- Find the most specific source type, S_X, of the operators in U:
  - If any of the operators in U convert from S, then S_X is S.
  - Otherwise, if any of the operators in U convert from types that encompass S, then S_X is the most encompassed type in the combined set of source types of those operators. If no most encompassed type can be found, then the conversion is ambiguous and a compile-time error occurs.
  - Otherwise, S_X is the most encompassing type in the combined set of source types of the operators in U. If no most encompassing type can be found, then the conversion is ambiguous and a compile-time error occurs.
- Find the most specific target type, T_X, of the operators in U:
  - If any of the operators in U convert to T, then T_X is T.
  - Otherwise, if any of the operators in U convert to types that are encompassed by T, then T_X is the most encompassing type in the combined set of source types of those operators. If no most encompassing type can be found, then the conversion is ambiguous and a compile-time error occurs.
  - Otherwise, T_X is the most encompassed type in the combined set of target types of the operators in U. If no most encompassed type can be found, then the conversion is ambiguous and a compile-time error occurs.
- If U contains exactly one user-defined conversion operator that converts from S_X to T_X, then this is the most specific conversion operator. If no such operator exists, or if more than one such operator exists, then the conversion is ambiguous and a compile-time error occurs. Otherwise, the user-defined conversion is applied:
  - If S is not S_X, then a standard explicit conversion from S to S_X is performed.
  - The most specific user-defined conversion operator is invoked to convert from S_X to T_X.
  - If T_X is not T, then a standard explicit conversion from T_X to T is performed.

14. Expressions

An expression is a sequence of operators and operands. This chapter defines the syntax, order of evaluation of operands and operators, and meaning of expressions.

14.1 Expression classifications

An expression is classified as one of the following:

- A value. Every value has an associated type.
- A variable. Every variable has an associated type, namely the declared type of the variable.
- A namespace. An expression with this classification can only appear as the left-hand side of a *member-access* (§14.5.4). In any other context, an expression classified as a namespace causes a compile-time error.
- A type. An expression with this classification can only appear as the left-hand side of a *member-access* (§14.5.4), or as an operand for the `as` operator (§14.9.10), the `is` operator (§14.9.9), or the `typeof` operator (§14.5.11). In any other context, an expression classified as a type causes a compile-time error.
- A method group, which is a set of overloaded methods resulting from a member lookup (§14.3). A method group may have an associated instance expression. When an instance method is invoked, the result of evaluating the instance expression becomes the instance represented by `this` (§14.5.7). A method group is only permitted in an *invocation-expression* (§14.5.5) or a *delegate-creation-expression* (§14.5.10.3). In any other context, an expression classified as a method group causes a compile-time error.
- A property access. Every property access has an associated type, namely the type of the property. Furthermore, a property access may have an associated instance expression. When an accessor (the `get` or `set` block) of an instance property access is invoked, the result of evaluating the instance expression becomes the instance represented by `this` (§14.5.7).
- An event access. Every event access has an associated type, namely the type of the event. Furthermore, an event access may have an associated instance expression. An event access may appear as the left-hand operand of the `+=` and `-=` operators (§14.13.3). In any other context, an expression classified as an event access causes a compile-time error.
- An indexer access. Every indexer access has an associated type, namely the element type of the indexer. Furthermore, an indexer access has an associated instance expression and an associated argument list. When an accessor (the `get` or `set` block) of an indexer access is invoked, the result of evaluating the instance expression becomes the instance represented by `this` (§14.5.7), and the result of evaluating the argument list becomes the parameter list of the invocation.
- Nothing. This occurs when the expression is an invocation of a method with a return type of `void`. An expression classified as nothing is only valid in the context of a *statement-expression* (§15.6).

The final result of an expression is never a namespace, type, method group, or event access. Rather, as noted above, these categories of expressions are intermediate constructs that are only permitted in certain contexts.

A property access or indexer access is always reclassified as a value by performing an invocation of the *get-accessor* or the *set-accessor*. The particular accessor is determined by the context of the property or indexer access: If the access is the target of an assignment, the *set-accessor* is invoked to assign a new value (§14.13.1). Otherwise, the *get-accessor* is invoked to obtain the current value (§14.1.1).

14.1.1 Values of expressions

Most of the constructs that involve an expression ultimately require the expression to denote a ***value***. In such cases, if the actual expression denotes a namespace, a type, a method group, or nothing, a compile-time error occurs. However, if the expression denotes a property access, an indexer access, or a variable, the value of the property, indexer, or variable is implicitly substituted:

- The value of a variable is simply the value currently stored in the storage location identified by the variable. A variable must be considered definitely assigned (§12.3) before its value can be obtained, or otherwise a compile-time error occurs.
- The value of a property access expression is obtained by invoking the *get-accessor* of the property. If the property has no *get-accessor*, a compile-time error occurs. Otherwise, a function member invocation (§14.4.3) is performed, and the result of the invocation becomes the value of the property access expression.
- The value of an indexer access expression is obtained by invoking the *get-accessor* of the indexer. If the indexer has no *get-accessor*, a compile-time error occurs. Otherwise, a function member invocation (§14.4.3) is performed with the argument list associated with the indexer access expression, and the result of the invocation becomes the value of the indexer access expression.

14.2 Operators

Expressions are constructed from ***operands*** and ***operators***. The operators of an expression indicate which operations to apply to the operands. Examples of operators include `+`, `-`, `*`, `/`, and `new`. Examples of operands include literals, fields, local variables, and expressions.

There are three kinds of operators:

- Unary operators. The unary operators take one operand and use either prefix notation (such as `-x`) or postfix notation (such as `x++`).
- Binary operators. The binary operators take two operands and all use infix notation (such as `x + y`).
- Ternary operator. Only one ternary operator, `?:`, exists; it takes three operands and uses infix notation (`c ? x : y`).

The order of evaluation of operators in an expression is determined by the ***precedence*** and ***associativity*** of the operators (§14.2.1).

The order in which operands in an expression are evaluated, is left to right. [*Example:* For example, in `F(i) + G(i++) * H(i)`, method `F` is called using the old value of `i`, then method `G` is called with the old value of `i`, and, finally, method `H` is called with the new value of `i`. This is separate from and unrelated to operator precedence. *end example*] Certain operators can be ***overloaded***. Operator overloading permits user-defined operator implementations to be specified for operations where one or both of the operands are of a user-defined class or struct type (§14.2.2).

14.2.1 Operator precedence and associativity

When an expression contains multiple operators, the ***precedence*** of the operators controls the order in which the individual operators are evaluated. [*Note:* For example, the expression `x + y * z` is evaluated as `x + (y * z)` because the `*` operator has higher precedence than the binary `+` operator. *end note*] The precedence of an operator is established by the definition of its associated grammar production. [*Note:* For example, an *additive-expression* consists of a sequence of *multiplicative-expression*s separated by `+` or `-` operators, thus giving the `+` and `-` operators lower precedence than the `*`, `/`, and `%` operators. *end note*]

The following table summarizes all operators in order of precedence from highest to lowest:

Section	Category	Operators
14.5	Primary	`x.y f(x) a[x] x++ x-- new typeof checked unchecked`
14.6	Unary	`+ - ! ~ ++x --x (T)x`
14.7	Multiplicative	`* / %`
14.7	Additive	`+ -`
14.8	Shift	`<< >>`
14.9	Relational and type-testing	`< > <= >= is as`
14.9	Equality	`== !=`
14.10	Logical AND	`&`
14.10	Logical XOR	`^`
14.10	Logical OR	`|`
14.11	Conditional AND	`&&`
14.11	Conditional OR	`||`
14.12	Conditional	`?:`
14.13	Assignment	`= *= /= %= += -= <<= >>= &= ^= |=`

When an operand occurs between two operators with the same precedence, the ***associativity*** of the operators controls the order in which the operations are performed:

- Except for the assignment operators, all binary operators are ***left-associative***, meaning that operations are performed from left to right. [*Example:* For example, `x + y + z` is evaluated as `(x + y) + z`. *end example*]

- The assignment operators and the conditional operator (`?:`) are ***right-associative***, meaning that operations are performed from right to left. [*Example:* For example, `x = y = z` is evaluated as `x = (y = z)`. *end example*]

Precedence and associativity can be controlled using parentheses. [*Example:* For example, `x + y * z` first multiplies `y` by `z` and then adds the result to `x`, but `(x + y) * z` first adds `x` and `y` and then multiplies the result by `z`. *end example*]

14.2.2 Operator overloading

All unary and binary operators have predefined implementations that are automatically available in any expression. In addition to the predefined implementations, user-defined implementations can be introduced by including `operator` declarations in classes and structs (§17.9). User-defined operator implementations always take precedence over predefined operator implementations: Only when no applicable user-defined operator implementations exist will the predefined operator implementations be considered.

The ***overloadable unary operators*** are:

```
+   -   !   ~   ++   --   true   false
```

[*Note*: Although `true` and `false` are not used explicitly in expressions, they are considered operators because they are invoked in several expression contexts: boolean expressions (§14.16) and expressions involving the conditional (§14.12), and conditional logical operators (§14.11). *end note*]

The ***overloadable binary operators*** are:

```
+   -   *   /   %   &   |   ^   <<   >>   ==   !=   >   <   >=   <=
```

Only the operators listed above can be overloaded. In particular, it is not possible to overload member access, method invocation, or the =, &&, ||, ?:, checked, unchecked, new, typeof, as, and is operators.

When a binary operator is overloaded, the corresponding assignment operator, if any, is also implicitly overloaded. For example, an overload of operator * is also an overload of operator *=. This is described further in §14.13. Note that the assignment operator itself (=) cannot be overloaded. An assignment always performs a simple bit-wise copy of a value into a variable.

Cast operations, such as (T)x, are overloaded by providing user-defined conversions (§13.4).

Element access, such as a[x], is not considered an overloadable operator. Instead, user-defined indexing is supported through indexers (§17.8).

In expressions, operators are referenced using operator notation, and in declarations, operators are referenced using functional notation. The following table shows the relationship between operator and functional notations for unary and binary operators. In the first entry, *op* denotes any overloadable unary prefix operator. In the second entry, *op* denotes the unary postfix ++ and -- operators. In the third entry, *op* denotes any overloadable binary operator. [*Note:* For an example of overloading the ++ and -- operators see §17.9.1. *end note*]

Operator notation	**Functional notation**
op x	operator *op*(x)
x *op*	operator *op*(x)
x *op* y	operator *op*(x, y)

User-defined operator declarations always require at least one of the parameters to be of the class or struct type that contains the operator declaration. [*Note:* Thus, it is not possible for a user-defined operator to have the same signature as a predefined operator. *end note*]

User-defined operator declarations cannot modify the syntax, precedence, or associativity of an operator. [*Example:* For example, the / operator is always a binary operator, always has the precedence level specified in §14.2.1, and is always left-associative. *end example*]

[*Note:*While it is possible for a user-defined operator to perform any computation it pleases, implementations that produce results other than those that are intuitively expected are strongly discouraged. For example, an implementation of operator == should compare the two operands for equality and return an appropriate bool result. *end note*]

The descriptions of individual operators in §14.5 through §14.13 specify the predefined implementations of the operators and any additional rules that apply to each operator. The descriptions make use of the terms ***unary operator overload resolution***, ***binary operator overload resolution***, and ***numeric promotion***, definitions of which are found in the following sections.

14.2.3 Unary operator overload resolution

An operation of the form *op* x or x *op*, where *op* is an overloadable unary operator, and x is an expression of type X, is processed as follows:

- The set of candidate user-defined operators provided by `X` for the operation `operator` *op*`(x)` is determined using the rules of §14.2.5.

- If the set of candidate user-defined operators is not empty, then this becomes the set of candidate operators for the operation. Otherwise, the predefined unary `operator` *op* implementations become the set of candidate operators for the operation. The predefined implementations of a given operator are specified in the description of the operator (§14.5 and §14.6).

- The overload resolution rules of §14.4.2 are applied to the set of candidate operators to select the best operator with respect to the argument list `(x)`, and this operator becomes the result of the overload resolution process. If overload resolution fails to select a single best operator, a compile-time error occurs.

14.2.4 Binary operator overload resolution

An operation of the form `x` *op* `y`, where *op* is an overloadable binary operator, `x` is an expression of type `X`, and `y` is an expression of type `Y`, is processed as follows:

- The set of candidate user-defined operators provided by `X` and `Y` for the operation `operator` *op*`(x, y)` is determined. The set consists of the union of the candidate operators provided by `X` and the candidate operators provided by `Y`, each determined using the rules of §14.2.5. If `X` and `Y` are the same type, or if `X` and `Y` are derived from a common base type, then shared candidate operators only occur in the combined set once.

- If the set of candidate user-defined operators is not empty, then this becomes the set of candidate operators for the operation. Otherwise, the predefined binary `operator` *op* implementations become the set of candidate operators for the operation. The predefined implementations of a given operator are specified in the description of the operator (§14.7 through §14.13).

- The overload resolution rules of §14.4.2 are applied to the set of candidate operators to select the best operator with respect to the argument list `(x, y)`, and this operator becomes the result of the overload resolution process. If overload resolution fails to select a single best operator, a compile-time error occurs.

14.2.5 Candidate user-defined operators

Given a type `T` and an operation `operator` *op*`(A)`, where *op* is an overloadable operator and `A` is an argument list, the set of candidate user-defined operators provided by `T` for `operator` *op*`(A)` is determined as follows:

- For all `operator` *op* declarations in `T`, if at least one operator is applicable (§14.4.2.1) with respect to the argument list `A`, then the set of candidate operators consists of all applicable `operator` *op* declarations in `T`.

- Otherwise, if `T` is `object`, the set of candidate operators is empty.

- Otherwise, the set of candidate operators provided by `T` is the set of candidate operators provided by the direct base class of `T`.

14.2.6 Numeric promotions

This clause is informative.

Numeric promotion consists of automatically performing certain implicit conversions of the operands of the predefined unary and binary numeric operators. Numeric promotion is not a distinct mechanism, but rather an effect of applying overload resolution to the predefined operators. Numeric promotion specifically does not affect evaluation of user-defined operators, although user-defined operators can be implemented to exhibit similar effects.

As an example of numeric promotion, consider the predefined implementations of the binary `*` operator:

```
int operator *(int x, int y);
uint operator *(uint x, uint y);
long operator *(long x, long y);
ulong operator *(ulong x, ulong y);
float operator *(float x, float y);
double operator *(double x, double y);
decimal operator *(decimal x, decimal y);
```

When overload resolution rules (§14.4.2) are applied to this set of operators, the effect is to select the first of the operators for which implicit conversions exist from the operand types. [*Example:* For example, for the operation `b * s`, where `b` is a `byte` and `s` is a `short`, overload resolution selects `operator *(int, int)` as the best operator. Thus, the effect is that `b` and `s` are converted to `int`, and the type of the result is `int`. Likewise, for the operation `i * d`, where `i` is an `int` and `d` is a `double`, overload resolution selects `operator *(double, double)` as the best operator. *end example*]

End of informative text.

14.2.6.1 Unary numeric promotions

This clause is informative.

Unary numeric promotion occurs for the operands of the predefined +, –, and ~ unary operators. Unary numeric promotion simply consists of converting operands of type `sbyte`, `byte`, `short`, `ushort`, or `char` to type `int`. Additionally, for the unary – operator, unary numeric promotion converts operands of type `uint` to type `long`.

End of informative text.

14.2.6.2 Binary numeric promotions

This clause is informative.

Binary numeric promotion occurs for the operands of the predefined +, –, *, /, %, &, |, ^, ==, !=, >, <, >=, and <= binary operators. Binary numeric promotion implicitly converts both operands to a common type which, in case of the non-relational operators, also becomes the result type of the operation. Binary numeric promotion consists of applying the following rules, in the order they appear here:

- If either operand is of type `decimal`, the other operand is converted to type `decimal`, or a compile-time error occurs if the other operand is of type `float` or `double`.
- Otherwise, if either operand is of type `double`, the other operand is converted to type `double`.
- Otherwise, if either operand is of type `float`, the other operand is converted to type `float`.
- Otherwise, if either operand is of type `ulong`, the other operand is converted to type `ulong`, or a compile-time error occurs if the other operand is of type `sbyte`, `short`, `int`, or `long`.
- Otherwise, if either operand is of type `long`, the other operand is converted to type `long`.
- Otherwise, if either operand is of type `uint` and the other operand is of type `sbyte`, `short`, or `int`, both operands are converted to type `long`.
- Otherwise, if either operand is of type `uint`, the other operand is converted to type `uint`.
- Otherwise, both operands are converted to type `int`.

[*Note:* Note that the first rule disallows any operations that mix the `decimal` type with the `double` and `float` types. The rule follows from the fact that there are no implicit conversions between the `decimal` type and the `double` and `float` types. *end note*]

[*Note:* Also note that it is not possible for an operand to be of type `ulong` when the other operand is of a signed integral type. The reason is that no integral type exists that can represent the full range of `ulong` as well as the signed integral types. *end note*]

In both of the above cases, a cast expression can be used to explicitly convert one operand to a type that is compatible with the other operand.

[*Example:* In the example

```
decimal AddPercent(decimal x, double percent) {
   return x * (1.0 + percent / 100.0);
}
```

a compile-time error occurs because a `decimal` cannot be multiplied by a `double`. The error is resolved by explicitly converting the second operand to `decimal`, `as follows`:

```
decimal AddPercent(decimal x, double percent) {
   return x * (decimal)(1.0 + percent / 100.0);
}
```

end example]

End of informative text.

14.3 Member lookup

A member lookup is the process whereby the meaning of a name in the context of a type is determined. A member lookup may occur as part of evaluating a *simple-name* (§14.5.2) or a *member-access* (§14.5.4) in an expression.

A member lookup of a name `N` in a type `T` is processed as follows:

- First, the set of all accessible (§10.5) members named `N` declared in `T` and the base types (§14.3.1) of `T` is constructed. Declarations that include an `override` modifier are excluded from the set. If no members named `N` exist and are accessible, then the lookup produces no match, and the following steps are not evaluated.
- Next, members that are hidden by other members are removed from the set. For every member `S.M` in the set, where `S` is the type in which the member `M` is declared, the following rules are applied:
  - If `M` is a constant, field, property, event, type, or enumeration member, then all members declared in a base type of `S` are removed from the set.
  - If `M` is a method, then all non-method members declared in a base type of `S` are removed from the set, and all methods with the same signature as `M` declared in a base type of `S` are removed from the set.
- Finally, having removed hidden members, the result of the lookup is determined:
  - If the set consists of a single non-method member, then this member is the result of the lookup.
  - Otherwise, if the set contains only methods, then this group of methods is the result of the lookup.
  - Otherwise, the lookup is ambiguous, and a compile-time error occurs (this situation can only occur for a member lookup in an interface that has multiple direct base interfaces).

For member lookups in types other than interfaces, and member lookups in interfaces that are strictly single-inheritance (each interface in the inheritance chain has exactly zero or one direct base interface), the effect of the lookup rules is simply that derived members hide base members with the same name or signature. Such single-inheritance lookups are never ambiguous. The ambiguities that can possibly arise from member lookups in multiple-inheritance interfaces are described in §20.2.5.

14.3.1 Base types

For purposes of member lookup, a type `T` is considered to have the following base types:

- If `T` is `object`, then `T` has no base type.
- If `T` is a *value-type*, the base type of `T` is the class type `object`.
- If `T` is a *class-type*, the base types of `T` are the base classes of T, including the class type `object`.
- If `T` is an *interface-type*, the base types of `T` are the base interfaces of `T` and the class type `object`.
- If `T` is an *array-type*, the base types of `T` are the class types `System.Array` and `object`.
- If `T` is a *delegate-type*, the base types of `T` are the class types `System.Delegate` and `object`.

14.4 Function members

Function members are members that contain executable statements. Function members are always members of types and cannot be members of namespaces. C# defines the following categories of function members:

- Methods
- Properties
- Events
- Indexers
- User-defined operators
- Instance constructors
- Static constructors
- Destructors

Except for static constructors and destructors (which cannot be invoked explicitly), the statements contained in function members are executed through function member invocations. The actual syntax for writing a function member invocation depends on the particular function member category.

The argument list (§14.4.1) of a function member invocation provides actual values or variable references for the parameters of the function member.

Invocations of methods, indexers, operators, and instance constructors employ overload resolution to determine which of a candidate set of function members to invoke. This process is described in §14.4.2.

Once a particular function member has been identified at compile-time, possibly through overload resolution, the actual run-time process of invoking the function member is described in §14.4.3.

[*Note:* The following table summarizes the processing that takes place in constructs involving the six categories of function members that can be explicitly invoked. In the table, `e`, `x`, `y`, and `value` indicate expressions classified as variables or values, `T` indicates an expression classified as a type, `F` is the simple name of a method, and `P` is the simple name of a property.

Construct	Example	Description
Method invocation	`F(x, y)`	Overload resolution is applied to select the best method `F` in the containing class or struct. The method is invoked with the argument list `(x, y)`. If the method is not `static`, the instance expression is `this`.
	`T.F(x, y)`	Overload resolution is applied to select the best method `F` in the class or struct T. A compile-time error occurs if the method is not `static`. The method is invoked with the argument list `(x, y)`.

Construct	Example	Description
	`e.F(x, y)`	Overload resolution is applied to select the best method F in the class, struct, or interface given by the type of `e`. A compile-time error occurs if the method is `static`. The method is invoked with the instance expression `e` and the argument list `(x, y)`.
Property access	`P`	The `get` accessor of the property `P` in the containing class or struct is invoked. A compile-time error occurs if `P` is write-only. If `P` is not `static`, the instance expression is `this`.
	`P = value`	The `set` accessor of the property `P` in the containing class or struct is invoked with the argument list `(value)`. A compile-time error occurs if `P` is read-only. If `P` is not `static`, the instance expression is `this`.
	`T.P`	The `get` accessor of the property `P` in the class or struct `T` is invoked. A compile-time error occurs if `P` is not `static` or if `P` is write-only.
	`T.P = value`	The `set` accessor of the property `P` in the class or struct `T` is invoked with the argument list `(value)`. A compile-time error occurs if `P` is not `static` or if `P` is read-only.
	`e.P`	The `get` accessor of the property `P` in the class, struct, or interface given by the type of `e` is invoked with the instance expression `e`. A compile-time error occurs if `P` is `static` or if `P` is write-only.
	`e.P = value`	The `set` accessor of the property `P` in the class, struct, or interface given by the type of `e` is invoked with the instance expression `e` and the argument list `(value)`. A compile-time error occurs if `P` is `static` or if `P` is read-only.
Event access	`E += value`	The `add` accessor of the event `E` in the containing class or struct is invoked. If `E` is not `static`, the instance expression is `this`.
	`E -= value`	The `remove` accessor of the event `E` in the containing class or struct is invoked. If `E` is not `static`, the instance expression is `this`.
	`T.E += value`	The `add` accessor of the event `E` in the class or struct `T` is invoked. A compile-time error occurs if `E` is not `static`.
	`T.E -= value`	The `remove` accessor of the event `E` in the class or struct `T` is invoked. A compile-time error occurs if `E` is not `static`.
	`e.E += value`	The `add` accessor of the event `E` in the class, struct, or interface given by the type of `e` is invoked with the instance expression `e`. A compile-time error occurs if `E` is `static`.
	`e.E -= value`	The `remove` accessor of the event `E` in the class, struct, or interface given by the type of `e` is invoked with the instance expression `e`. A compile-time error occurs if `E` is `static`.
Indexer access	`e[x, y]`	Overload resolution is applied to select the best indexer in the class, struct, or interface given by the type of e. The `get` accessor of the indexer is invoked with the instance expression `e` and the argument list `(x, y)`. A compile-time error occurs if the indexer is write-only.

Construct	Example	Description
	`e[x, y] = value`	Overload resolution is applied to select the best indexer in the class, struct, or interface given by the type of `e`. The `set` accessor of the indexer is invoked with the instance expression `e` and the argument list `(x, y, value)`. A compile-time error occurs if the indexer is read-only.
Operator invocation	`-x`	Overload resolution is applied to select the best unary operator in the class or struct given by the type of `x`. The selected operator is invoked with the argument list `(x)`.
	`x + y`	Overload resolution is applied to select the best binary operator in the classes or structs given by the types of x and y. The selected operator is invoked with the argument list `(x, y)`.
Instance constructor invocation	`new T(x, y)`	Overload resolution is applied to select the best instance constructor in the class or struct T. The instance constructor is invoked with the argument list `(x, y)`.

end note]

14.4.1 Argument lists

Every function member invocation includes an argument list, which provides actual values or variable references for the parameters of the function member. The syntax for specifying the argument list of a function member invocation depends on the function member category:

- For instance constructors, methods, and delegates, the arguments are specified as an *argument-list*, as described below.
- For properties, the argument list is empty when invoking the `get` accessor, and consists of the expression specified as the right operand of the assignment operator when invoking the `set` accessor.
- For events, the argument list consists of the expression specified as the right operand of the `+=` or `-=` operator.
- For indexers, the argument list consists of the expressions specified between the square brackets in the indexer access. When invoking the `set` accessor, the argument list additionally includes the expression specified as the right operand of the assignment operator.
- For user-defined operators, the argument list consists of the single operand of the unary operator or the two operands of the binary operator.

The arguments of properties (§17.6), events (§17.7), indexers (§17.8), and user-defined operators (§17.9) are always passed as value parameters (§17.5.1.1). Reference and output parameters are not supported for these categories of function members.

The arguments of an instance constructor, method, or delegate invocation are specified as an *argument-list*:

argument-list:
    argument
    argument-list `,` *argument*

argument:
    expression
    `ref` *variable-reference*
    `out` *variable-reference*

An *argument-list* consists of one or more *argument*s, separated by commas. Each argument can take one of the following forms:

- An *expression*, indicating that the argument is passed as a value parameter (§17.5.1.1).
- The keyword `ref` followed by a *variable-reference* (§12.3.3), indicating that the argument is passed as a reference parameter (§17.5.1.2). A variable must be definitely assigned (§12.3) before it can be passed as a reference parameter. A volatile field (§17.4.3) cannot be passed as a reference parameter.
- The keyword `out` followed by a *variable-reference* (§12.3.3), indicating that the argument is passed as an output parameter (§17.5.1.3). A variable is considered definitely assigned (§12.3) following a function member invocation in which the variable is passed as an output parameter. A volatile field (§17.4.3) cannot be passed as an output parameter.

During the run-time processing of a function member invocation (§14.4.3), the expressions or variable references of an argument list are evaluated in order, from left to right, as follows:

- For a value parameter, the argument expression is evaluated and an implicit conversion (§13.1) to the corresponding parameter type is performed. The resulting value becomes the initial value of the value parameter in the function member invocation.
- For a reference or output parameter, the variable reference is evaluated and the resulting storage location becomes the storage location represented by the parameter in the function member invocation. If the variable reference given as a reference or output parameter is an array element of a *reference-type*, a run-time check is performed to ensure that the element type of the array is identical to the type of the parameter. If this check fails, a `System.ArrayTypeMismatchException` is thrown.

Methods, indexers, and instance constructors may declare their right-most parameter to be a parameter array (§17.5.1.4). Such function members are invoked either in their normal form or in their expanded form depending on which is applicable (§14.4.2.1):

- When a function member with a parameter array is invoked in its normal form, the argument given for the parameter array must be a single expression of a type that is implicitly convertible (§13.1) to the parameter array type. In this case, the parameter array acts precisely like a value parameter.
- When a function member with a parameter array is invoked in its expanded form, the invocation must specify zero or more arguments for the parameter array, where each argument is an expression of a type that is implicitly convertible (§13.1) to the element type of the parameter array. In this case, the invocation creates an instance of the parameter array type with a length corresponding to the number of arguments, initializes the elements of the array instance with the given argument values, and uses the newly created array instance as the actual argument.

The expressions of an argument list are always evaluated in the order they are written. [*Example:* Thus, the example

```
class Test
{
   static void F(int x, int y, int z) {
      System.Console.WriteLine("x = {0}, y = {1}, z = {2}", x, y, z);
   }

   static void Main() {
      int i = 0;
      F(i++, i++, i++);
   }
}
```

produces the output

```
x = 0, y = 1, z = 2
```

end example]

The array covariance rules (§19.5) permit a value of an array type `A[]` to be a reference to an instance of an array type `B[]`, provided an implicit reference conversion exists from `B` to A. Because of these rules, when an array element of a *reference-type* is passed as a reference or output parameter, a run-time check is

required to ensure that the actual element type of the array is *identical* to that of the parameter. [*Example:* In the example

```
class Test
{
   static void F(ref object x) {…}

   static void Main() {
      object[] a = new object[10];
      object[] b = new string[10];
      F(ref a[0]);      // Ok
      F(ref b[1]);      // ArrayTypeMismatchException
   }
}
```

the second invocation of `F` causes a `System.ArrayTypeMismatchException` to be thrown because the actual element type of `b` is `string` and not `object`. *end example*]

When a function member with a parameter array is invoked in its expanded form, the invocation is processed exactly as if an array creation expression with an array initializer (§14.5.10.2) was inserted around the expanded parameters. [*Example:* For example, given the declaration

```
void F(int x, int y, params object[] args);
```

the following invocations of the expanded form of the method

```
F(10, 20);
F(10, 20, 30, 40);
F(10, 20, 1, "hello", 3.0);
```

correspond exactly to

```
F(10, 20, new object[] {});
F(10, 20, new object[] {30, 40});
F(10, 20, new object[] {1, "hello", 3.0});
```

end example] In particular, note that an empty array is created when there are zero arguments given for the parameter array.

14.4.2 Overload resolution

Overload resolution is a compile-time mechanism for selecting the best function member to invoke given an argument list and a set of candidate function members. Overload resolution selects the function member to invoke in the following distinct contexts within C#:

- Invocation of a method named in an *invocation-expression* (§14.5.5).
- Invocation of an instance constructor named in an *object-creation-expression* (§14.5.10.1).
- Invocation of an indexer accessor through an *element-access* (§14.5.6).
- Invocation of a predefined or user-defined operator referenced in an expression (§14.2.3 and §14.2.4).

Each of these contexts defines the set of candidate function members and the list of arguments in its own unique way. However, once the candidate function members and the argument list have been identified, the selection of the best function member is the same in all cases:

- First, the set of candidate function members is reduced to those function members that are applicable with respect to the given argument list (§14.4.2.1). If this reduced set is empty, a compile-time error occurs.

- Then, given the set of applicable candidate function members, the best function member in that set is located. If the set contains only one function member, then that function member is the best function member. Otherwise, the best function member is the one function member that is better than all other function members with respect to the given argument list, provided that each function member is compared to all other function members using the rules in §14.4.2.2. If there is not exactly one function member that is better than all other function members, then the function member invocation is ambiguous and a compile-time error occurs.

The following sections define the exact meanings of the terms ***applicable function member*** and ***better function member***.

14.4.2.1 Applicable function member

A function member is said to be an ***applicable function member*** with respect to an argument list `A` when all of the following are true:

- The number of arguments in `A` is identical to the number of parameters in the function member declaration.

- For each argument in A, the parameter passing mode of the argument (i.e., value, `ref`, or `out`) is identical to the parameter passing mode of the corresponding parameter, and

  - for a value parameter or a parameter array, an implicit conversion (§13.1) exists from the type of the argument to the type of the corresponding parameter, or

  - for a `ref` or `out` parameter, the type of the argument is identical to the type of the corresponding parameter. [*Note:* After all, a `ref` or `out` parameter is an alias for the argument passed. *end note*]

For a function member that includes a parameter array, if the function member is applicable by the above rules, it is said to be applicable in its ***normal form***. If a function member that includes a parameter array is not applicable in its normal form, the function member may instead be applicable in its ***expanded form***:

- The expanded form is constructed by replacing the parameter array in the function member declaration with zero or more value parameters of the element type of the parameter array such that the number of arguments in the argument list `A` matches the total number of parameters. If `A` has fewer arguments than the number of fixed parameters in the function member declaration, the expanded form of the function member cannot be constructed and is thus not applicable.

- If the class, struct, or interface in which the function member is declared already contains another applicable function member with the same signature as the expanded form, the expanded form is not applicable.

- Otherwise, the expanded form is applicable if for each argument in `A` the parameter passing mode of the argument is identical to the parameter passing mode of the corresponding parameter, and

  - for a fixed value parameter or a value parameter created by the expansion, an implicit conversion (§13.1) exists from the type of the argument to the type of the corresponding parameter, or

  - for a `ref` or `out` parameter, the type of the argument is identical to the type of the corresponding parameter.

14.4.2.2 Better function member

Given an argument list `A` with a set of argument types A_1, A_2, …, A_N and two applicable function members M_P and M_Q with parameter types P_1, P_2, …, P_N and Q_1, Q_2, …, Q_N, M_P is defined to be a ***better function member*** than M_Q if

- for each argument, the implicit conversion from A_X to P_X is not worse than the implicit conversion from A_X to Q_X, and
- for at least one argument, the conversion from A_X to P_X is better than the conversion from A_X to Q_X.

When performing this evaluation, if M_P or M_Q is applicable in its expanded form, then P_X or Q_X refers to a parameter in the expanded form of the parameter list.

14.4.2.3 Better conversion

Given an implicit conversion C_1 that converts from a type `S` to a type T_1, and an implicit conversion C_2 that converts from a type `S` to a type T_2, the ***better conversion*** of the two conversions is determined as follows:

- If T_1 and T_2 are the same type, neither conversion is better.
- If `S` is T_1, C_1 is the better conversion.
- If `S` is T_2, C_2 is the better conversion.
- If an implicit conversion from T_1 to T_2 exists, and no implicit conversion from T_2 to T_1 exists, C_1 is the better conversion.
- If an implicit conversion from T_2 to T_1 exists, and no implicit conversion from T_1 to T_2 exists, C_2 is the better conversion.
- If T_1 is `sbyte` and T_2 is `byte`, `ushort`, `uint`, or `ulong`, C_1 is the better conversion.
- If T_2 is `sbyte` and T_1 is `byte`, `ushort`, `uint`, or `ulong`, C_2 is the better conversion.
- If T_1 is `short` and T_2 is `ushort`, `uint`, or `ulong`, C_1 is the better conversion.
- If T_2 is `short` and T_1 is `ushort`, `uint`, or `ulong`, C_2 is the better conversion.
- If T_1 is `int` and T_2 is `uint`, or `ulong`, C_1 is the better conversion.
- If T_2 is `int` and T_1 is `uint`, or `ulong`, C_2 is the better conversion.
- If T_1 is `long` and T_2 is `ulong`, C_1 is the better conversion.
- If T_2 is `long` and T_1 is `ulong`, C_2 is the better conversion.
- Otherwise, neither conversion is better.

If an implicit conversion C_1 is defined by these rules to be a better conversion than an implicit conversion C_2, then it is also the case that C_2 is a ***worse conversion*** than C_1.

14.4.3 Function member invocation

This section describes the process that takes place at run-time to invoke a particular function member. It is assumed that a compile-time process has already determined the particular member to invoke, possibly by applying overload resolution to a set of candidate function members.

For purposes of describing the invocation process, function members are divided into two categories:

- Static function members. These are static methods, instance constructors, static property accessors, and user-defined operators. Static function members are always non-virtual.
- Instance function members. These are instance methods, instance property accessors, and indexer accessors. Instance function members are either non-virtual or virtual, and are always invoked on a particular instance. The instance is computed by an instance expression, and it becomes accessible within the function member as `this` (§14.5.7).

The run-time processing of a function member invocation consists of the following steps, where `M` is the function member and, if `M` is an instance member, `E` is the instance expression:

- If `M` is a static function member:
  - The argument list is evaluated as described in §14.4.1.
  - `M` is invoked.
- If `M` is an instance function member declared in a *value-type*:
  - `E` is evaluated. If this evaluation causes an exception, then no further steps are executed.
  - If `E` is not classified as a variable, then a temporary local variable of `E`'s type is created and the value of `E` is assigned to that variable. `E` is then reclassified as a reference to that temporary local variable. The temporary variable is accessible as `this` within M, but not in any other way. Thus, only when `E` is a true variable is it possible for the caller to observe the changes that `M` makes to `this`.
  - The argument list is evaluated as described in §14.4.1.
  - `M` is invoked. The variable referenced by `E` becomes the variable referenced by `this`.
- If `M` is an instance function member declared in a *reference-type*:
  - `E` is evaluated. If this evaluation causes an exception, then no further steps are executed.
  - The argument list is evaluated as described in §14.4.1.
  - If the type of `E` is a *value-type*, a boxing conversion (§11.3.1) is performed to convert `E` to type `object`, and `E` is considered to be of type `object` in the following steps. [*Note:* In this case, `M` could only be a member of `System.Object`. *end note*]
  - The value of `E` is checked to be valid. If the value of `E` is `null`, a `System.NullReferenceException` is thrown and no further steps are executed.
  - The function member implementation to invoke is determined:
    - If the compile-time type of `E` is an interface, the function member to invoke is the implementation of `M` provided by the run-time type of the instance referenced by `E`. This function member is determined by applying the interface mapping rules (§20.4.2) to determine the implementation of `M` provided by the run-time type of the instance referenced by `E`.
    - Otherwise, if `M` is a virtual function member, the function member to invoke is the implementation of `M` provided by the run-time type of the instance referenced by `E`. This function member is determined by applying the rules for determining the most derived implementation (§17.5.3) of `M` with respect to the run-time type of the instance referenced by `E`.
    - Otherwise, `M` is a non-virtual function member, and the function member to invoke is `M` itself.
  - The function member implementation determined in the step above is invoked. The object referenced by `E` becomes the object referenced by `this`.

14.4.3.1 Invocations on boxed instances

A function member implemented in a *value-type* can be invoked through a boxed instance of that *value-type* in the following situations:

- When the function member is an `override` of a method inherited from type `object` and is invoked through an instance expression of type `object`.
- When the function member is an implementation of an interface function member and is invoked through an instance expression of an *interface-type*.
- When the function member is invoked through a delegate.

In these situations, the boxed instance is considered to contain a variable of the *value-type*, and this variable becomes the variable referenced by `this` within the function member invocation. [*Note:* In particular, this means that when a function member is invoked on a boxed instance, it is possible for the function member to modify the value contained in the boxed instance. *end note*]

14.5 Primary expressions

Primary expressions include the simplest forms of expressions.

primary-expression:
array-creation-expression
primary-no-array-creation-expression

primary-no-array-creation-expression:
literal
simple-name
parenthesized-expression
member-access
invocation-expression
element-access
this-access
base-access
post-increment-expression
post-decrement-expression
object-creation-expression
delegate-creation-expression
typeof-expression
sizeof-expression
checked-expression
unchecked-expression

Primary expressions are divided between *array-creation-expression*s and *primary-no-array-creation-expression*s. Treating *array-creation-expression* in this way, rather than listing it along with the other simple expression forms, enables the grammar to disallow potentially confusing code such as

```
object o = new int[3][1];
```

which would otherwise be interpreted as

```
object o = (new int[3])[1];
```

14.5.1 Literals

A *primary-expression* that consists of a *literal* (§9.4.4) is classified as a value.

14.5.2 Simple names

A *simple-name* consists of a single identifier.

simple-name:
identifier

A *simple-name* is evaluated and classified as follows:

- If the *simple-name* appears within a *block* and if the *block*'s (or an enclosing block's) local variable declaration space (§10.3) contains a local variable or parameter with the given name, then the *simple-name* refers to that local variable or parameter and is classified as a variable.

- Otherwise, for each type T, starting with the immediately enclosing class, struct, or enumeration declaration and continuing with each enclosing outer class or struct declaration (if any), if a member lookup of the *simple-name* in `T` produces a match:

  - If `T` is the immediately enclosing class or struct type and the lookup identifies one or more methods, the result is a method group with an associated instance expression of `this`.

  - If `T` is the immediately enclosing class or struct type, if the lookup identifies an instance member, and if the reference occurs within the *block* of an instance constructor, an instance method, or an

instance accessor, the result is the same as a member access (§14.5.4) of the form `this.E`, where `E` is the *simple-name*.

- Otherwise, the result is the same as a member access (§14.5.4) of the form `T.E`, where `E` is the *simple-name*. In this case, it is a compile-time error for the *simple-name* to refer to an instance member.

- Otherwise, starting with the namespace in which the *simple-name* occurs, continuing with each enclosing namespace (if any), and ending with the global namespace, the following steps are evaluated until an entity is located:
  - If the namespace contains a namespace member with the given name, then the *simple-name* refers to that member and, depending on the member, is classified as a namespace or a type.
  - Otherwise, if the namespace has a corresponding namespace declaration enclosing the location where the *simple-name* occurs, then:
    - If the namespace declaration contains a *using-alias-directive* that associates the given name with an imported namespace or type, then the *simple-name* refers to that namespace or type.
    - Otherwise, if the namespaces imported by the *using-namespace-directive*s of the namespace declaration contain exactly one type with the given name, then the *simple-name* refers to that type.
    - Otherwise, if the namespaces imported by the *using-namespace-directive*s of the namespace declaration contain more than one type with the given name, then the *simple-name* is ambiguous and a compile-time error occurs.
- Otherwise, the name given by the *simple-name* is undefined and a compile-time error occurs.

14.5.2.1 Invariant meaning in blocks

For each occurrence of a given identifier as a *simple-name* in an expression, every other occurrence of the same identifier as a *simple-name* in an expression within the immediately enclosing *block* (§15.2) or *switch-block* (§15.7.2) must refer to the same entity. This rule ensures that the meaning of a name in the context of an expression is always the same within a block.

The example

```
class Test
{
   double x;
   void F(bool b) {
      x = 1.0;
      if (b) {
         int x = 1;
      }
   }
}
```

results in a compile-time error because `x` refers to different entities within the outer block (the extent of which includes the nested block in the `if` statement). In contrast, the example

```
class Test
{
   double x;
   void F(bool b) {
      if (b) {
         x = 1.0;
      }
      else {
         int x = 1;
      }
   }
}
```

is permitted because the name `x` is never used in the outer block.

Note that the rule of invariant meaning applies only to simple names. It is perfectly valid for the same identifier to have one meaning as a simple name and another meaning as right operand of a member access (§14.5.4). [*Example:* For example:

```
struct Point
{
   int x, y;

   public Point(int x, int y) {
      this.x = x;
      this.y = y;
   }
}
```

The example above illustrates a common pattern of using the names of fields as parameter names in an instance constructor. In the example, the simple names `x` and `y` refer to the parameters, but that does not prevent the member access expressions `this.x` and `this.y` from accessing the fields. *end example*]

14.5.3 Parenthesized expressions

A *parenthesized-expression* consists of an *expression* enclosed in parentheses.

parenthesized-expression:
`(` *expression* `)`

A *parenthesized-expression* is evaluated by evaluating the *expression* within the parentheses. If the *expression* within the parentheses denotes a namespace, type, or method group, a compile-time error occurs. Otherwise, the result of the *parenthesized-expression* is the result of the evaluation of the contained *expression*.

14.5.4 Member access

A *member-access* consists of a *primary-expression* or a *predefined-type*, followed by a "`.`" token, followed by an *identifier*.

member-access:
primary-expression `.` *identifier*
predefined-type `.` *identifier*

predefined-type: one of

```
bool     byte     char     decimal  double   float    int      long
object   sbyte    short    string   uint     ulong    ushort
```

A *member-access* of the form `E.I`, where `E` is a *primary-expression* or a *predefined-type* and `I` is an *identifier*, is evaluated and classified as follows:

- If `E` is a namespace and `I` is the name of an accessible member of that namespace, then the result is that member and, depending on the member, is classified as a namespace or a type.
- If `E` is a *predefined-type* or a *primary-expression* classified as a type, and a member lookup (§14.3) of `I` in `E` produces a match, then `E.I` is evaluated and classified as follows:
  - If `I` identifies a type, then the result is that type.
  - If `I` identifies one or more methods, then the result is a method group with no associated instance expression.
  - If `I` identifies a `static` property, then the result is a property access with no associated instance expression.
  - If `I` identifies a `static` field:

- If the field is `readonly` and the reference occurs outside the static constructor of the class or struct in which the field is declared, then the result is a value, namely the value of the static field `I` in E.
- Otherwise, the result is a variable, namely the static field `I` in E.

- If `I` identifies a `static` event:
  - If the reference occurs within the class or struct in which the event is declared, and the event was declared without *event-accessor-declaration*s (§17.7), then `E.I` is processed exactly as if `I` was a static field.
  - Otherwise, the result is an event access with no associated instance expression.
- If `I` identifies a constant, then the result is a value, namely the value of that constant.
- If `I` identifies an enumeration member, then the result is a value, namely the value of that enumeration member.
- Otherwise, `E.I` is an invalid member reference, and a compile-time error occurs.

- If `E` is a property access, indexer access, variable, or value, the type of which is T, and a member lookup (§14.3) of `I` in `T` produces a match, then `E.I` is evaluated and classified as follows:
  - First, if `E` is a property or indexer access, then the value of the property or indexer access is obtained (§14.1.1) and `E` is reclassified as a value.
  - If `I` identifies one or more methods, then the result is a method group with an associated instance expression of E.
  - If `I` identifies an instance property, then the result is a property access with an associated instance expression of E.
  - If `T` is a *class-type* and `I` identifies an instance field of that *class-type*:
    - If the value of `E` is `null`, then a `System.NullReferenceException` is thrown.
    - Otherwise, if the field is `readonly` and the reference occurs outside an instance constructor of the class in which the field is declared, then the result is a value, namely the value of the field `I` in the object referenced by E.
    - Otherwise, the result is a variable, namely the field `I` in the object referenced by E.
  - If `T` is a *struct-type* and `I` identifies an instance field of that *struct-type*:
    - If `E` is a value, or if the field is `readonly` and the reference occurs outside an instance constructor of the struct in which the field is declared, then the result is a value, namely the value of the field `I` in the struct instance given by E.
    - Otherwise, the result is a variable, namely the field `I` in the struct instance given by E.
  - If `I` identifies an instance event:
    - If the reference occurs within the class or struct in which the event is declared, and the event was declared without *event-accessor-declaration*s (§17.7), then `E.I` is processed exactly as if `I` was an instance field.
    - Otherwise, the result is an event access with an associated instance expression of E.
- Otherwise, `E.I` is an invalid member reference, and a compile-time error occurs.

14.5.4.1 Identical simple names and type names

In a member access of the form `E.I`, if `E` is a single identifier, and if the meaning of `E` as a *simple-name* (§14.5.2) is a constant, field, property, local variable, or parameter with the same type as the meaning of `E` as a *type-name* (§10.8), then both possible meanings of `E` are permitted. The two possible meanings of `E.I` are

never ambiguous, since `I` must necessarily be a member of the type `E` in both cases. In other words, the rule simply permits access to the static members of `E` where a compile-time error would otherwise have occurred. [*Example:* For example:

```
struct Color
{
   public static readonly Color White = new Color(…);
   public static readonly Color Black = new Color(…);

   public Color Complement() {…}
}

class A
{
   public Color Color;              // Field Color of type Color

   void F() {
      Color = Color.Black;          // References Color.Black static
member
      Color = Color.Complement();   // Invokes Complement() on Color
field
   }

   static void G() {
      Color c = Color.White;        // References Color.White static
member
   }
}
```

Within the `A` class, those occurrences of the `Color` identifier that reference the `Color` type are underlined, and those that reference the `Color` field are not underlined. *end example*]

14.5.5 Invocation expressions

An *invocation-expression* is used to invoke a method.

invocation-expression:
primary-expression (*argument-list*$_{opt}$)

The *primary-expression* of an *invocation-expression* must be a method group or a value of a *delegate-type*. If the *primary-expression* is a method group, the *invocation-expression* is a method invocation (§14.5.5.1). If the *primary-expression* is a value of a *delegate-type*, the *invocation-expression* is a delegate invocation (§14.5.5.2). If the *primary-expression* is neither a method group nor a value of a *delegate-type*, a compile-time error occurs.

The optional *argument-list* (§14.4.1) provides values or variable references for the parameters of the method.

The result of evaluating an *invocation-expression* is classified as follows:

- If the *invocation-expression* invokes a method or delegate that returns `void`, the result is nothing. An expression that is classified as nothing cannot be an operand of any operator, and is permitted only in the context of a *statement-expression* (§15.6).
- Otherwise, the result is a value of the type returned by the method or delegate.

14.5.5.1 Method invocations

For a method invocation, the *primary-expression* of the *invocation-expression* must be a method group. The method group identifies the one method to invoke or the set of overloaded methods from which to choose a specific method to invoke. In the latter case, determination of the specific method to invoke is based on the context provided by the types of the arguments in the *argument-list*.

The compile-time processing of a method invocation of the form `M(A)`, where `M` is a method group and `A` is an optional *argument-list*, consists of the following steps:

- The set of candidate methods for the method invocation is constructed. Starting with the set of methods associated with M, which were found by a previous member lookup (§14.3), the set is reduced to those

methods that are applicable with respect to the argument list A. The set reduction consists of applying the following rules to each method T.N in the set, where T is the type in which the method N is declared:

- If N is not applicable with respect to A (§14.4.2.1), then N is removed from the set.
- If N is applicable with respect to A (§14.4.2.1), then all methods declared in a base type of T are removed from the set.

- If the resulting set of candidate methods is empty, then no applicable methods exist, and a compile-time error occurs. If the candidate methods are not all declared in the same type, the method invocation is ambiguous, and a compile-time error occurs (this latter situation can only occur for an invocation of a method in an interface that has multiple direct base interfaces, as described in §20.2.5).

- The best method of the set of candidate methods is identified using the overload resolution rules of §14.4.2. If a single best method cannot be identified, the method invocation is ambiguous, and a compile-time error occurs.

- Given a best method, the invocation of the method is validated in the context of the method group: If the best method is a static method, the method group must have resulted from a *simple-name* or a *member-access* through a type. If the best method is an instance method, the method group must have resulted from a *simple-name*, a *member-access* through a variable or value, or a *base-access*. If neither of these requirements are true, a compile-time error occurs.

Once a method has been selected and validated at compile-time by the above steps, the actual run-time invocation is processed according to the rules of function member invocation described in §14.4.3.

[*Note:* The intuitive effect of the resolution rules described above is as follows: To locate the particular method invoked by a method invocation, start with the type indicated by the method invocation and proceed up the inheritance chain until at least one applicable, accessible, non-override method declaration is found. Then perform overload resolution on the set of applicable, accessible, non-override methods declared in that type and invoke the method thus selected. *end note*]

14.5.5.2 Delegate invocations

For a delegate invocation, the *primary-expression* of the *invocation-expression* must be a value of a *delegate-type*. Furthermore, considering the *delegate-type* to be a function member with the same parameter list as the *delegate-type*, the *delegate-type* must be applicable (§14.4.2.1) with respect to the *argument-list* of the *invocation-expression*.

The run-time processing of a delegate invocation of the form D(A), where D is a *primary-expression* of a *delegate-type* and A is an optional *argument-list*, consists of the following steps:

- D is evaluated. If this evaluation causes an exception, no further steps are executed.

- The value of D is checked to be valid. If the value of D is null, a System.NullReferenceException is thrown and no further steps are executed.

- Otherwise, D is a reference to a delegate instance. A function member invocation (§14.4.3) is performed on the method referenced by the delegate. If the method is an instance method, the instance of the invocation becomes the instance referenced by the delegate.

14.5.6 Element access

An *element-access* consists of a *primary-no-array-creation-expression*, followed by a "[" token, followed by an *expression-list*, followed by a "]" token. The *expression-list* consists of one or more *expression*s, separated by commas.

element-access:
    primary-no-array-creation-expression [*expression-list*]

expression-list:
expression
expression-list , *expression*

If the *primary-no-array-creation-expression* of an *element-access* is a value of an *array-type*, the *element-access* is an array access (§14.5.6.1). Otherwise, the *primary-no-array-creation-expression* must be a variable or value of a class, struct, or interface type that has one or more indexer members, in which case the *element-access* is an indexer access (§14.5.6.2).

14.5.6.1 Array access

For an array access, the *primary-no-array-creation-expression* of the *element-access* must be a value of an *array-type*. The number of expressions in the *expression-list* must be the same as the rank of the *array-type*, and each expression must be of type `int`, `uint`, `long`, `ulong`, or of a type that can be implicitly converted to one or more of these types.

The result of evaluating an array access is a variable of the element type of the array, namely the array element selected by the value(s) of the expression(s) in the *expression-list*.

The run-time processing of an array access of the form `P[A]`, where `P` is a *primary-no-array-creation-expression* of an *array-type* and `A` is an *expression-list*, consists of the following steps:

- `P` is evaluated. If this evaluation causes an exception, no further steps are executed.
- The index expressions of the *expression-list* are evaluated in order, from left to right. Following evaluation of each index expression, an implicit conversion (§13.1) to one of the following types is performed: `int`, `uint`, `long`, `ulong`. The first type in this list for which an implicit conversion exists is chosen. For instance, if the index expression is of type `short` then an implicit conversion to `int` is performed, since implicit conversions from `short` to `int` and from `short` to `long` are possible. If evaluation of an index expression or the subsequent implicit conversion causes an exception, then no further index expressions are evaluated and no further steps are executed.
- The value of `P` is checked to be valid. If the value of `P` is `null`, a `System.NullReferenceException` is thrown and no further steps are executed.
- The value of each expression in the *expression-list* is checked against the actual bounds of each dimension of the array instance referenced by P. If one or more values are out of range, a `System.IndexOutOfRangeException` is thrown and no further steps are executed.
- The location of the array element given by the index expression(s) is computed, and this location becomes the result of the array access.

14.5.6.2 Indexer access

For an indexer access, the *primary-no-array-creation-expression* of the *element-access* must be a variable or value of a class, struct, or interface type, and this type must implement one or more indexers that are applicable with respect to the *expression-list* of the *element-access*.

The compile-time processing of an indexer access of the form `P[A]`, where `P` is a *primary-no-array-creation-expression* of a class, struct, or interface type `T`, and `A` is an *expression-list*, consists of the following steps:

- The set of indexers provided by `T` is constructed. The set consists of all indexers declared in `T` or a base type of `T` that are not `override` declarations and are accessible in the current context (§10.5).
- The set is reduced to those indexers that are applicable and not hidden by other indexers. The following rules are applied to each indexer `S.I` in the set, where `S` is the type in which the indexer `I` is declared:
    - If `I` is not applicable with respect to `A` (§14.4.2.1), then `I` is removed from the set.
    - If `I` is applicable with respect to `A` (§14.4.2.1), then all indexers declared in a base type of `S` are removed from the set.

- If the resulting set of candidate indexers is empty, then no applicable indexers exist, and a compile-time error occurs. If the candidate indexers are not all declared in the same type, the indexer access is ambiguous, and a compile-time error occurs (this latter situation can only occur for an indexer access on an instance of an interface that has multiple direct base interfaces).

- The best indexer of the set of candidate indexers is identified using the overload resolution rules of §14.4.2. If a single best indexer cannot be identified, the indexer access is ambiguous, and a compile-time error occurs.

- The index expressions of the *expression-list* are evaluated in order, from left to right. The result of processing the indexer access is an expression classified as an indexer access. The indexer access expression references the indexer determined in the step above, and has an associated instance expression of `P` and an associated argument list of A.

Depending on the context in which it is used, an indexer access causes invocation of either the *get-accessor* or the *set-accessor* of the indexer. If the indexer access is the target of an assignment, the *set-accessor* is invoked to assign a new value (§14.13.1). In all other cases, the *get-accessor* is invoked to obtain the current value (§14.1.1).

14.5.7 This access

A *this-access* consists of the reserved word `this`.

this-access:
    `this`

A *this-access* is permitted only in the *block* of an instance constructor, an instance method, or an instance accessor. It has one of the following meanings:

- When `this` is used in a *primary-expression* within an instance constructor of a class, it is classified as a value. The type of the value is the class within which the usage occurs, and the value is a reference to the object being constructed.

- When `this` is used in a *primary-expression* within an instance method or instance accessor of a class, it is classified as a value. The type of the value is the class within which the usage occurs, and the value is a reference to the object for which the method or accessor was invoked.

- When `this` is used in a *primary-expression* within an instance constructor of a struct, it is classified as a variable. The type of the variable is the struct within which the usage occurs, and the variable represents the struct being constructed. The `this` variable of an instance constructor of a struct behaves exactly the same as an `out` parameter of the struct type—in particular, this means that the variable must be definitely assigned in every execution path of the instance constructor.

- When `this` is used in a *primary-expression* within an instance method or instance accessor of a struct, it is classified as a variable. The type of the variable is the struct within which the usage occurs, and the variable represents the struct for which the method or accessor was invoked. The `this` variable of an instance method of a struct behaves exactly the same as a `ref` parameter of the struct type.

Use of `this` in a *primary-expression* in a context other than the ones listed above is a compile-time error. In particular, it is not possible to refer to `this` in a static method, a static property accessor, or in a *variable-initializer* of a field declaration.

14.5.8 Base access

A *base-access* consists of the reserved word `base` followed by either a "`.`" token and an identifier or an *expression-list* enclosed in square brackets:

base-access:
    `base` `.` *identifier*
    `base` `[` *expression-list* `]`

A *base-access* is used to access base class members that are hidden by similarly named members in the current class or struct. A *base-access* is permitted only in the *block* of an instance constructor, an instance method, or an instance accessor. When `base.I` occurs in a class or struct, `I` must denote a member of the base class of that class or struct. Likewise, when `base[E]` occurs in a class, an applicable indexer must exist in the base class.

At compile-time, *base-access* expressions of the form `base.I` and `base[E]` are evaluated exactly as if they were written `((B)this).I` and `((B)this)[E]`, where `B` is the base class of the class or struct in which the construct occurs. Thus, `base.I` and `base[E]` correspond to `this.I` and `this[E]`, except `this` is viewed as an instance of the base class.

When a *base-access* references a virtual function member (a method, property, or indexer), the determination of which function member to invoke at run-time (§14.4.3) is changed. The function member that is invoked is determined by finding the most derived implementation (§17.5.3) of the function member with respect to `B` (instead of with respect to the run-time type of `this`, as would be usual in a non-base access). Thus, within an `override` of a `virtual` function member, a *base-access* can be used to invoke the inherited implementation of the function member. If the function member referenced by a *base-access* is abstract, a compile-time error occurs.

14.5.9 Postfix increment and decrement operators

post-increment-expression:
primary-expression `++`

post-decrement-expression:
primary-expression `--`

The operand of a postfix increment or decrement operation must be an expression classified as a variable, a property access, or an indexer access. The result of the operation is a value of the same type as the operand.

If the operand of a postfix increment or decrement operation is a property or indexer access, the property or indexer must have both a `get` and a `set` accessor. If this is not the case, a compile-time error occurs.

Unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. Predefined `++` and `--` operators exist for the following types: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, `decimal`, and any enum type. The predefined `++` operators return the value produced by adding 1 to the operand, and the predefined `--` operators return the value produced by subtracting 1 from the operand.

The run-time processing of a postfix increment or decrement operation of the form `x++` or `x--` consists of the following steps:

- If `x` is classified as a variable:
  - `x` is evaluated to produce the variable.
  - The value of `x` is saved.
  - The selected operator is invoked with the saved value of `x` as its argument.
  - The value returned by the operator is stored in the location given by the evaluation of `x`.
  - The saved value of `x` becomes the result of the operation.
- If `x` is classified as a property or indexer access:
  - The instance expression (if `x` is not `static`) and the argument list (if `x` is an indexer access) associated with `x` are evaluated, and the results are used in the subsequent `get` and `set` accessor invocations.
  - The `get` accessor of `x` is invoked and the returned value is saved.
  - The selected operator is invoked with the saved value of `x` as its argument.
  - The `set` accessor of `x` is invoked with the value returned by the operator as its `value` argument.

- The saved value of `x` becomes the result of the operation.

The `++` and `--` operators also support prefix notation (§14.6.5). The result of `x++` or `x--` is the value of `x` *before* the operation, whereas the result of `++x` or `--x` is the value of `x` *after* the operation. In either case, `x` itself has the same value after the operation.

An `operator ++` or `operator --` implementation can be invoked using either postfix or prefix notation. It is not possible to have separate operator implementations for the two notations.

14.5.10 The `new` operator

The `new` operator is used to create new instances of types.

There are three forms of `new` expressions:

- Object creation expressions are used to create new instances of class types and value types.
- Array creation expressions are used to create new instances of array types.
- Delegate creation expressions are used to create new instances of delegate types.

The `new` operator implies creation of an instance of a type, but does not necessarily imply dynamic allocation of memory. In particular, instances of value types require no additional memory beyond the variables in which they reside, and no dynamic allocations occur when `new` is used to create instances of value types.

14.5.10.1 Object creation expressions

An *object-creation-expression* is used to create a new instance of a *class-type* or a *value-type*.

object-creation-expression:
`new` *type* `(` *argument-list*$_{opt}$ `)`

The *type* of an *object-creation-expression* must be a *class-type* or a *value-type*. The *type* cannot be an `abstract` *class-type*.

The optional *argument-list* (§14.4.1) is permitted only if the *type* is a *class-type* or a *struct-type*.

The compile-time processing of an *object-creation-expression* of the form `new T(A)`, where `T` is a *class-type* or a *value-type* and `A` is an optional *argument-list*, consists of the following steps:

- If `T` is a *value-type* and `A` is not present:
  - The *object-creation-expression* is a default constructor invocation. The result of the *object-creation-expression* is a value of type `T`, namely the default value for `T` as defined in §11.1.1.
- Otherwise, if `T` is a *class-type* or a *struct-type*:
  - If `T` is an `abstract` *class-type*, a compile-time error occurs.
  - The instance constructor to invoke is determined using the overload resolution rules of §14.4.2. The set of candidate instance constructors consists of all accessible instance constructors declared in T. If the set of candidate instance constructors is empty, or if a single best instance constructor cannot be identified, a compile-time error occurs.
  - The result of the *object-creation-expression* is a value of type `T`, namely the value produced by invoking the instance constructor determined in the step above.
- Otherwise, the *object-creation-expression* is invalid, and a compile-time error occurs.

The run-time processing of an *object-creation-expression* of the form `new T(A)`, where `T` is *class-type* or a *struct-type* and `A` is an optional *argument-list*, consists of the following steps:

- If `T` is a *class-type*:
  - A new instance of class `T` is allocated. If there is not enough memory available to allocate the new instance, a `System.OutOfMemoryException` is thrown and no further steps are executed.
  - All fields of the new instance are initialized to their default values (§12.2).
  - The instance constructor is invoked according to the rules of function member invocation (§14.4.3). A reference to the newly allocated instance is automatically passed to the instance constructor and the instance can be accessed from within that constructor as `this`.
- If `T` is a *struct-type*:
  - An instance of type `T` is created by allocating a temporary local variable. Since an instance constructor of a *struct-type* is required to definitely assign a value to each field of the instance being created, no initialization of the temporary variable is necessary.
  - The instance constructor is invoked according to the rules of function member invocation (§14.4.3). A reference to the newly allocated instance is automatically passed to the instance constructor and the instance can be accessed from within that constructor as `this`.

14.5.10.2 Array creation expressions

An *array-creation-expression* is used to create a new instance of an *array-type*.

array-creation-expression:
`new` *non-array-type* `[` *expression-list* `]` *rank-specifiers*$_{opt}$ *array-initializer*$_{opt}$
`new` *array-type* *array-initializer*

An array creation expression of the first form allocates an array instance of the type that results from deleting each of the individual expressions from the expression list. For example, the array creation expression `new int[10,20]` produces an array instance of type `int[,]`, and the array creation expression `new int[10][,]` produces an array of type `int[][,]`. Each expression in the expression list must be of type `int`, `uint`, `long`, or `ulong`, or of a type that can be implicitly converted to one or more of these types. The value of each expression determines the length of the corresponding dimension in the newly allocated array instance. Since the length of an array dimension must be nonnegative, it is a compile-time error to have a constant expression with a negative value, in the expression list.

Except in an unsafe context (§25.1), the layout of arrays is unspecified.

If an array creation expression of the first form includes an array initializer, each expression in the expression list must be a constant and the rank and dimension lengths specified by the expression list must match those of the array initializer.
In an array creation expression of the second form, the rank of the specified array type must match that of the array initializer. The individual dimension lengths are inferred from the number of elements in each of the corresponding nesting levels of the array initializer. Thus, the expression

```
new int[,] {{0, 1}, {2, 3}, {4, 5}}
```

exactly corresponds to

```
new int[3, 2] {{0, 1}, {2, 3}, {4, 5}}
```

Array initializers are described further in §19.6.

The result of evaluating an array creation expression is classified as a value, namely a reference to the newly allocated array instance. The run-time processing of an array creation expression consists of the following steps:

- The dimension length expressions of the *expression-list* are evaluated in order, from left to right. Following evaluation of each expression, an implicit conversion (§13.1) to one of the following types is performed: `int`, `uint`, `long`, `ulong`. The first type in this list for which an implicit conversion exists is chosen. If evaluation of an expression or the subsequent implicit conversion causes an exception, then no further expressions are evaluated and no further steps are executed.

- The computed values for the dimension lengths are validated, as follows: If one or more of the values are less than zero, a `System.OverflowException` is thrown and no further steps are executed.

- An array instance with the given dimension lengths is allocated. If there is not enough memory available to allocate the new instance, a `System.OutOfMemoryException` is thrown and no further steps are executed.

- All elements of the new array instance are initialized to their default values (§12.2).

- If the array creation expression contains an array initializer, then each expression in the array initializer is evaluated and assigned to its corresponding array element. The evaluations and assignments are performed in the order the expressions are written in the array initializer—in other words, elements are initialized in increasing index order, with the rightmost dimension increasing first. If evaluation of a given expression or the subsequent assignment to the corresponding array element causes an exception, then no further elements are initialized (and the remaining elements will thus have their default values).

An array creation expression permits instantiation of an array with elements of an array type, but the elements of such an array must be manually initialized. [*Example:* For example, the statement

```
int[][] a = new int[100][];
```

creates a single-dimensional array with 100 elements of type `int[]`. The initial value of each element is `null`. *end example*] It is not possible for the same array creation expression to also instantiate the sub-arrays, and the statement

```
int[][] a = new int[100][5];     // Error
```

results in a compile-time error. Instantiation of the sub-arrays must instead be performed manually, as in

```
int[][] a = new int[100][];
for (int i = 0; i < 100; i++) a[i] = new int[5];
```

When an array of arrays has a “rectangular” shape, that is when the sub-arrays are all of the same length, it is more efficient to use a multi-dimensional array. In the example above, instantiation of the array of arrays creates 101 objects—one outer array and 100 sub-arrays. In contrast,

```
int[,] = new int[100, 5];
```

creates only a single object, a two-dimensional array, and accomplishes the allocation in a single statement.

14.5.10.3 Delegate creation expressions

A *delegate-creation-expression* is used to create a new instance of a *delegate-type*.

delegate-creation-expression:
`new` *delegate-type* `(` *expression* `)`

The argument of a delegate creation expression must be a method group (§14.1) or a value of a *delegate-type*. If the argument is a method group, it identifies the method and, for an instance method, the object for which to create a delegate. If the argument is a value of a *delegate-type*, it identifies a delegate instance of which to create a copy.

The compile-time processing of a *delegate-creation-expression* of the form `new D(E)`, where `D` is a *delegate-type* and `E` is an *expression*, consists of the following steps:

- If `E` is a method group:
  - The set of methods identified by `E` must include exactly one method that is compatible (§22.1) with `D`, and this method becomes the one to which the newly created delegate refers. If no matching

method exists, or if more than one matching method exists, a compile-time error occurs. If the selected method is an instance method, the instance expression associated with `E` determines the target object of the delegate.

  - As in a method invocation, the selected method must be compatible with the context of the method group: If the method is a static method, the method group must have resulted from a *simple-name* or a *member-access* through a type. If the method is an instance method, the method group must have resulted from a *simple-name* or a *member-access* through a variable or value. If the selected method does not match the context of the method group, a compile-time error occurs.
  - The result is a value of type `D`, namely a newly created delegate that refers to the selected method and target object.

- Otherwise, if `E` is a value of a *delegate-type*:
  - `D` and `E` must be compatible (§22.1); otherwise, a compile-time error occurs.
  - The result is a value of type `D`, namely a newly created delegate that refers to the same invocation list as `E`.
- Otherwise, the delegate creation expression is invalid, and a compile-time error occurs.

The run-time processing of a *delegate-creation-expression* of the form `new D(E)`, where `D` is a *delegate-type* and `E` is an *expression*, consists of the following steps:

- If `E` is a method group:
  - If the method selected at compile-time is a static method, the target object of the delegate is `null`. Otherwise, the selected method is an instance method, and the target object of the delegate is determined from the instance expression associated with `E`:
    - The instance expression is evaluated. If this evaluation causes an exception, no further steps are executed.
    - If the instance expression is of a *reference-type*, the value computed by the instance expression becomes the target object. If the target object is `null`, a `System.NullReferenceException` is thrown and no further steps are executed.
    - If the instance expression is of a *value-type*, a boxing operation (§11.3.1) is performed to convert the value to an object, and this object becomes the target object.
  - A new instance of the delegate type `D` is allocated. If there is not enough memory available to allocate the new instance, a `System.OutOfMemoryException` is thrown and no further steps are executed.
  - The new delegate instance is initialized with a reference to the method that was determined at compile-time and a reference to the target object computed above.
- If `E` is a value of a *delegate-type*:
  - `E` is evaluated. If this evaluation causes an exception, no further steps are executed.
  - If the value of `E` is `null`, a `System.NullReferenceException` is thrown and no further steps are executed.
  - A new instance of the delegate type `D` is allocated. If there is not enough memory available to allocate the new instance, a `System.OutOfMemoryException` is thrown and no further steps are executed.
  - The new delegate instance is initialized with references to the same invocation list as the delegate instance given by E.

The method and object to which a delegate refers are determined when the delegate is instantiated and then remain constant for the entire lifetime of the delegate. In other words, it is not possible to change the target method or object of a delegate once it has been created. [*Note:* Remember, when two delegates are

combined or one is removed from another, a new delegate results; no existing delegate has its content changed. *end note*]

It is not possible to create a delegate that refers to a property, indexer, user-defined operator, instance constructor, destructor, or static constructor.

[*Example:* As described above, when a delegate is created from a method group, the formal parameter list and return type of the delegate determine which of the overloaded methods to select. In the example

```
delegate double DoubleFunc(double x);

class A
{
   DoubleFunc f = new DoubleFunc(Square);

   static float Square(float x) {
      return x * x;
   }

   static double Square(double x) {
      return x * x;
   }
}
```

the `A.f` field is initialized with a delegate that refers to the second `Square` method because that method exactly matches the formal parameter list and return type of `DoubleFunc`. Had the second `Square` method not been present, a compile-time error would have occurred. *end example*]

14.5.11 The `typeof` operator

The `typeof` operator is used to obtain the `System.Type` object for a type.

typeof-expression:
`typeof (` *type* `)`
`typeof ( void )`

The first form of *typeof-expression* consists of a `typeof` keyword followed by a parenthesized *type*. The result of an expression of this form is the `System.Type` object for the indicated type. There is only one `System.Type` object for any given type. [*Note:* This means that for type `T`, `typeof(T) == typeof(T)` is always true. *end note*]

The second form of *typeof-expression* consists of a `typeof` keyword followed by a parenthesized `void` keyword. The result of an expression of this form is the `System.Type` object that represents the absence of a type. The type object returned by `typeof(void)` is distinct from the type object returned for any type. [*Note:* This special type object is useful in class libraries that allow reflection onto methods in the language, where those methods wish to have a way to represent the return type of any method, including void methods, with an instance of `System.Type`. *end note*]

[*Example:* The example

```
using System;
class Test
{
   static void Main() {
      Type[] t = {
         typeof(int),
         typeof(System.Int32),
         typeof(string),
         typeof(double[]),
         typeof(void)      };
      for (int i = 0; i < t.Length; i++) {
         Console.WriteLine(t[i].FullName);
      }
   }
}
```

produces the following output:

```
System.Int32
System.Int32
System.String
System.Double[]
System.Void
```

Note that `int` and `System.Int32` are the same type. *end example*]

14.5.12 The checked and unchecked operators

The `checked` and `unchecked` operators are used to control the ***overflow checking context*** for integral-type arithmetic operations and conversions.

checked-expression:
`checked` `(` *expression* `)`

unchecked-expression:
`unchecked` `(` *expression* `)`

The `checked` operator evaluates the contained expression in a checked context, and the `unchecked` operator evaluates the contained expression in an unchecked context. A *checked-expression* or *unchecked-expression* corresponds exactly to a *parenthesized-expression* (§14.5.3), except that the contained expression is evaluated in the given overflow checking context.

The overflow checking context can also be controlled through the `checked` and `unchecked` statements (§15.11).

The following operations are affected by the overflow checking context established by the `checked` and `unchecked` operators and statements:

- The predefined `++` and `--` unary operators (§14.5.9 and §14.6.5), when the operand is of an integral type.
- The predefined `-` unary operator (§14.6.2), when the operand is of an integral type.
- The predefined `+`, `-`, `*`, and `/` binary operators (§14.7), when both operands are of integral types.
- Explicit numeric conversions (§13.2.1) from one integral type to another integral type.

When one of the above operations produce a result that is too large to represent in the destination type, the context in which the operation is performed controls the resulting behavior:

- In a `checked` context, if the operation is a constant expression (§14.15), a compile-time error occurs. Otherwise, when the operation is performed at run-time, a `System.OverflowException` is thrown.
- In an `unchecked` context, the result is truncated by discarding any high-order bits that do not fit in the destination type.

For non-constant expressions (expressions that are evaluated at run-time) that are not enclosed by any `checked` or `unchecked` operators or statements, the default overflow checking context is `unchecked`, unless external factors (such as compiler switches and execution environment configuration) call for `checked` evaluation.

For constant expressions (expressions that can be fully evaluated at compile-time), the default overflow checking context is always `checked`. Unless a constant expression is explicitly placed in an `unchecked` context, overflows that occur during the compile-time evaluation of the expression always cause compile-time errors.

[*Note:* Developers may benefit if they exercise their code using checked mode (as well as unchecked mode). It also seems reasonable that, unless otherwise requested, the default overflow checking context is set to checked when debugging is enabled. *end note*]

[*Example:* In the example

```
class Test
{
   static readonly int x = 1000000;
   static readonly int y = 1000000;

   static int F() {
      return checked(x * y);     // Throws OverflowException
   }

   static int G() {
      return unchecked(x * y);   // Returns -727379968
   }

   static int H() {
      return x * y;              // Depends on default
   }
}
```

no compile-time errors are reported since neither of the expressions can be evaluated at compile-time. At run-time, the `F` method throws a `System.OverflowException`, and the `G` method returns –727379968 (the lower 32 bits of the out-of-range result). The behavior of the `H` method depends on the default overflow checking context for the compilation, but it is either the same as `F` or the same as `G`. *end example*]

[*Example:* In the example

```
class Test
{
   const int x = 1000000;
   const int y = 1000000;

   static int F() {
      return checked(x * y);     // Compile error, overflow
   }

   static int G() {
      return unchecked(x * y);   // Returns -727379968
   }

   static int H() {
      return x * y;              // Compile error, overflow
   }
}
```

the overflows that occur when evaluating the constant expressions in `F` and `H` cause compile-time errors to be reported because the expressions are evaluated in a `checked` context. An overflow also occurs when evaluating the constant expression in `G`, but since the evaluation takes place in an `unchecked` context, the overflow is not reported. *end example*]

The `checked` and `unchecked` operators only affect the overflow checking context for those operations that are textually contained within the "`(`" and "`)`" tokens. The operators have no effect on function members that are invoked as a result of evaluating the contained expression. [*Example:* In the example

```
class Test
{
   static int Multiply(int x, int y) {
      return x * y;
   }

   static int F() {
      return checked(Multiply(1000000, 1000000));
   }
}
```

the use of `checked` in `F` does not affect the evaluation of `x * y` in `Multiply`, so `x * y` is evaluated in the default overflow checking context. *end example*]

The `unchecked` operator is convenient when writing constants of the signed integral types in hexadecimal notation. [*Example:* For example:

```
class Test
{
   public const int AllBits = unchecked((int)0xFFFFFFFF);
   public const int HighBit = unchecked((int)0x80000000);
}
```

Both of the hexadecimal constants above are of type `uint`. Because the constants are outside the `int` range, without the `unchecked` operator, the casts to `int` would produce compile-time errors. *end example*]

[*Note:* The `checked` and `unchecked` operators and statements allow programmers to control certain aspects of some numeric calculations. However, the behavior of some numeric operators depends on their operands' data types. For example, multiplying two decimals always results in an exception on overflow *even* within an explicitly `unchecked` construct. Similarly, multiplying two floats never results in an exception on overflow *even* within an explicitly `checked` construct. In addition, other operators are *never* affected by the mode of checking, whether default or explicit. As a service to programmers, it is recommended that the compiler issue a warning when there is an arithmetic expression within an explicitly `checked` or `unchecked` context (by operator or statement) that cannot possibly be affected by the specified mode of checking. Since such a warning is not required, the compiler has flexibility in determining the circumstances that merit the issuance of such warnings. *end note*]

14.6 Unary expressions

unary-expression:
primary-expression
`+` *unary-expression*
`-` *unary-expression*
`!` *unary-expression*
`~` *unary-expression*
`*` *unary-expression*
`&` *unary-expression*
pre-increment-expression
pre-decrement-expression
cast-expression

14.6.1 Unary plus operator

For an operation of the form `+x`, unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. The operand is converted to the parameter type of the selected operator, and the type of the result is the return type of the operator. The predefined unary plus operators are:

```
int operator +(int x);
uint operator +(uint x);
long operator +(long x);
ulong operator +(ulong x);
float operator +(float x);
double operator +(double x);
decimal operator +(decimal x);
```

For each of these operators, the result is simply the value of the operand.

14.6.2 Unary minus operator

For an operation of the form `-x`, unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. The operand is converted to the parameter type of the selected operator, and the type of the result is the return type of the operator. The predefined negation operators are:

- Integer negation:

```
int operator -(int x);
long operator -(long x);
```

  The result is computed by subtracting `x` from zero. In a `checked` context, if the value of `x` is the maximum negative `int` or `long`, a `System.OverflowException` is thrown. In an `unchecked`

context, if the value of `x` is the maximum negative `int` or `long`, the result is that same value and the overflow is not reported.

If the operand of the negation operator is of type `uint`, it is converted to type `long`, and the type of the result is `long`. An exception is the rule that permits the `int` value −2147483648 (-2^{31}) to be written as a decimal integer literal (§9.4.4.2).

If the operand of the negation operator is of type `ulong`, a compile-time error occurs. An exception is the rule that permits the `long` value −9223372036854775808 (-2^{63}) to be written as a decimal integer literal (§9.4.4.2).

- Floating-point negation:

```
float operator –(float x);
double operator –(double x);
```

The result is the value of `x` with its sign inverted. If `x` is NaN, the result is also NaN.

- Decimal negation:

```
decimal operator –(decimal x);
```

The result is computed by subtracting `x` from zero.

Decimal negation is equivalent to using the unary minus operator of type `System.Decimal`.

14.6.3 Logical negation operator

For an operation of the form `!x`, unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. The operand is converted to the parameter type of the selected operator, and the type of the result is the return type of the operator. Only one predefined logical negation operator exists:

```
bool operator !(bool x);
```

This operator computes the logical negation of the operand: If the operand is `true`, the result is `false`. If the operand is `false`, the result is `true`.

14.6.4 Bitwise complement operator

For an operation of the form `~x`, unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. The operand is converted to the parameter type of the selected operator, and the type of the result is the return type of the operator. The predefined bitwise complement operators are:

```
int operator ~(int x);
uint operator ~(uint x);
long operator ~(long x);
ulong operator ~(ulong x);
```

For each of these operators, the result of the operation is the bitwise complement of `x`.

Every enumeration type `E` implicitly provides the following bitwise complement operator:

```
E operator ~(E x);
```

The result of evaluating `~x`, where `x` is an expression of an enumeration type `E` with an underlying type `U`, is exactly the same as evaluating `(E)(~(U)x)`.

14.6.5 Prefix increment and decrement operators

pre-increment-expression:
`++` *unary-expression*

pre-decrement-expression:
`--` *unary-expression*

The operand of a prefix increment or decrement operation must be an expression classified as a variable, a property access, or an indexer access. The result of the operation is a value of the same type as the operand.

If the operand of a prefix increment or decrement operation is a property or indexer access, the property or indexer must have both a `get` and a `set` accessor. If this is not the case, a compile-time error occurs.

Unary operator overload resolution (§14.2.3) is applied to select a specific operator implementation. Predefined `++` and `--` operators exist for the following types: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, `decimal`, and any enum type. The predefined `++` operators return the value produced by adding 1 to the operand, and the predefined `--` operators return the value produced by subtracting 1 from the operand.

The run-time processing of a prefix increment or decrement operation of the form `++x` or `--x` consists of the following steps:

- If `x` is classified as a variable:
  - `x` is evaluated to produce the variable.
  - The selected operator is invoked with the value of `x` as its argument.
  - The value returned by the operator is stored in the location given by the evaluation of `x`.
  - The value returned by the operator becomes the result of the operation.
- If `x` is classified as a property or indexer access:
  - The instance expression (if `x` is not `static`) and the argument list (if `x` is an indexer access) associated with `x` are evaluated, and the results are used in the subsequent `get` and `set` accessor invocations.
  - The `get` accessor of `x` is invoked.
  - The selected operator is invoked with the value returned by the `get` accessor as its argument.
  - The `set` accessor of `x` is invoked with the value returned by the operator as its `value` argument.
  - The value returned by the operator becomes the result of the operation.

The `++` and `--` operators also support postfix notation (§14.5.9). The result of `x++` or `x--` is the value of `x` *before* the operation, whereas the result of `++x` or `--x` is the value of `x` *after* the operation. In either case, `x` itself has the same value after the operation.

An `operator ++` or `operator --` implementation can be invoked using either postfix or prefix notation. It is not possible to have separate operator implementations for the two notations.

14.6.6 Cast expressions

A *cast-expression* is used to explicitly convert an expression to a given type.

cast-expression:
`(` *type* `)` *unary-expression*

A *cast-expression* of the form `(T)E`, where `T` is a *type* and `E` is a *unary-expression*, performs an explicit conversion (§13.2) of the value of `E` to type `T`. If no explicit conversion exists from the type of `E` to `T`, a compile-time error occurs. Otherwise, the result is the value produced by the explicit conversion. The result is always classified as a value, even if `E` denotes a variable.

The grammar for a *cast-expression* leads to certain syntactic ambiguities. For example, the expression `(x)-y` could either be interpreted as a *cast-expression* (a cast of `-y` to type `x`) or as an *additive-expression* combined with a *parenthesized-expression* (which computes the value `x - y`).

To resolve *cast-expression* ambiguities, the following rule exists: A sequence of one or more *token*s (§9.4) enclosed in parentheses is considered the start of a *cast-expression* only if at least one of the following are true:

- The sequence of tokens is correct grammar for a *type*, but not for an *expression*.
- The sequence of tokens is correct grammar for a *type*, and the token immediately following the closing parentheses is the token "~", the token "`!`", the token "`(`", an *identifier* (§9.4.1), a *literal* (§9.4.4), or any *keyword* (§9.4.3) except `as` and `is`.

[*Note:* The above rule means that only if the construct is unambiguously a *cast-expression* is it considered a *cast-expression*. *end note*]

The term "correct grammar" above means only that the sequence of tokens must conform to the particular grammatical production. It specifically does not consider the actual meaning of any constituent identifiers. For example, if `x` and `y` are identifiers, then `x.y` is correct grammar for a type, even if `x.y` doesn't actually denote a type.

[*Note:* From the disambiguation rule, it follows that, if `x` and `y` are identifiers, `(x)y`, `(x)(y)`, and `(x)(-y)` are *cast-expression*s, but `(x)-y` is not, even if `x` identifies a type. However, if `x` is a keyword that identifies a predefined type (such as `int`), then all four forms are *cast-expression*s (because such a keyword could not possibly be an expression by itself). *end note*]

14.7 Arithmetic operators

The `*`, `/`, `%`, `+`, and `–` operators are called the arithmetic operators.

multiplicative-expression:
unary-expression
multiplicative-expression `*` *unary-expression*
multiplicative-expression `/` *unary-expression*
multiplicative-expression `%` *unary-expression*

additive-expression:
multiplicative-expression
additive-expression `+` *multiplicative-expression*
additive-expression `–` *multiplicative-expression*

14.7.1 Multiplication operator

For an operation of the form `x * y`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined multiplication operators are listed below. The operators all compute the product of `x` and `y`.

- Integer multiplication:

```
int operator *(int x, int y);
uint operator *(uint x, uint y);
long operator *(long x, long y);
ulong operator *(ulong x, ulong y);
```

  In a `checked` context, if the product is outside the range of the result type, a `System.OverflowException` is thrown. In an `unchecked` context, overflows are not reported and any significant high-order bits outside the range of the result type are discarded.

- Floating-point multiplication:

```
float operator *(float x, float y);
double operator *(double x, double y);
```

  The product is computed according to the rules of IEEE 754 arithmetic. The following table lists the results of all possible combinations of nonzero finite values, zeros, infinities, and NaN's. In the table, `x` and `y` are positive finite values. `z` is the result of `x * y`. If the result is too large for the destination type, `z` is infinity. If the result is too small for the destination type, `z` is zero.

	+y	−y	+0	−0	+∞	−∞	NaN
+x	+z	−z	+0	−0	+∞	−∞	NaN
−x	−z	+z	−0	+0	−∞	+∞	NaN
+0	+0	−0	+0	−0	NaN	NaN	NaN
−0	−0	+0	−0	+0	NaN	NaN	NaN
+∞	+∞	−∞	NaN	NaN	+∞	−∞	NaN
−∞	−∞	+∞	NaN	NaN	−∞	+∞	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Decimal multiplication:

```
decimal operator *(decimal x, decimal y);
```

If the resulting value is too large to represent in the `decimal` format, a `System.OverflowException` is thrown. If the result value is too small to represent in the `decimal` format, the result is zero. The scale of the result, before any rounding, is the sum of the scales of the two operands.

Decimal multiplication is equivalent to using the multiplication operator of type `System.Decimal`.

14.7.2 Division operator

For an operation of the form `x / y`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined division operators are listed below. The operators all compute the quotient of `x` and `y`.

- Integer division:

```
int operator /(int x, int y);
uint operator /(uint x, uint y);
long operator /(long x, long y);
ulong operator /(ulong x, ulong y);
```

If the value of the right operand is zero, a `System.DivideByZeroException` is thrown.

The division rounds the result towards zero, and the absolute value of the result is the largest possible integer that is less than the absolute value of the quotient of the two operands. The result is zero or positive when the two operands have the same sign and zero or negative when the two operands have opposite signs.

If the left operand is the maximum negative `int` or `long` value and the right operand is `−1`, an overflow occurs. In a `checked` context, this causes a `System.OverflowException` to be thrown. In an `unchecked` context, the overflow is not reported and the result is instead the value of the left operand.

- Floating-point division:

```
float operator /(float x, float y);
double operator /(double x, double y);
```

The quotient is computed according to the rules of IEEE 754 arithmetic. The following table lists the results of all possible combinations of nonzero finite values, zeros, infinities, and NaN's. In the table, `x` and `y` are positive finite values. `z` is the result of `x / y`. If the result is too large for the destination type, `z` is infinity. If the result is too small for the destination type, `z` is zero.

	+y	−y	+0	−0	+∞	−∞	NaN
+x	+z	−z	+∞	−∞	+0	−0	NaN
−x	−z	+z	−∞	+∞	−0	+0	NaN
+0	+0	−0	NaN	NaN	+0	−0	NaN
−0	−0	+0	NaN	NaN	−0	+0	NaN
+∞	+∞	−∞	+∞	−∞	NaN	NaN	NaN
−∞	−∞	+∞	−∞	+∞	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Decimal division:

```
decimal operator /(decimal x, decimal y);
```

If the value of the right operand is zero, a `System.DivideByZeroException` is thrown. If the resulting value is too large to represent in the `decimal` format, a `System.OverflowException` is thrown. If the result value is too small to represent in the `decimal` format, the result is zero. The scale of the result, before any rounding, is the smallest scale that will preserve a result equal to the exact result.

Decimal division is equivalent to using the division operator of type `System.Decimal`.

14.7.3 Remainder operator

For an operation of the form `x % y`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined remainder operators are listed below. The operators all compute the remainder of the division between `x` and `y`.

- Integer remainder:

```
int operator %(int x, int y);
uint operator %(uint x, uint y);
long operator %(long x, long y);
ulong operator %(ulong x, ulong y);
```

The result of `x % y` is the value produced by `x - (x / y) * y`. If `y` is zero, a `System.DivideByZeroException` is thrown. The remainder operator never causes an overflow.

- Floating-point remainder:

```
float operator %(float x, float y);
double operator %(double x, double y);
```

The following table lists the results of all possible combinations of nonzero finite values, zeros, infinities, and NaN's. In the table, `x` and `y` are positive finite values. `z` is the result of `x % y` and is computed as `x - n * y`, where `n` is the largest possible integer that is less than or equal to `x / y`. This method of computing the remainder is analogous to that used for integer operands, but differs from the IEEE 754 definition (in which `n` is the integer closest to `x / y`).

	+y	−y	+0	−0	+∞	−∞	NaN
+x	+z	+z	NaN	NaN	x	x	NaN
−x	−z	−z	NaN	NaN	−x	−x	NaN
+0	+0	+0	NaN	NaN	+0	+0	NaN
−0	−0	−0	NaN	NaN	−0	−0	NaN
+∞	NaN	NaN	NaN	NaN	NaN	NaN	NaN
−∞	NaN	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Decimal remainder:

```
decimal operator %(decimal x, decimal y);
```

  If the value of the right operand is zero, a `System.DivideByZeroException` is thrown. If the resulting value is too large to represent in the `decimal` format, a `System.OverflowException` is thrown. If the result value is too small to represent in the `decimal` format, the result is zero. The scale of the result, before any rounding, is the same as the scale of `y`, and the sign of the result, if non-zero, is the same as that of `x`.

  Decimal remainder is equivalent to using the remainder operator of type `System.Decimal`.

14.7.4 Addition operator

For an operation of the form `x + y`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined addition operators are listed below. For numeric and enumeration types, the predefined addition operators compute the sum of the two operands. When one or both operands are of type `string`, the predefined addition operators concatenate the string representation of the operands.

- Integer addition:

```
int operator +(int x, int y);
uint operator +(uint x, uint y);
long operator +(long x, long y);
ulong operator +(ulong x, ulong y);
```

  In a `checked` context, if the sum is outside the range of the result type, a `System.OverflowException` is thrown. In an `unchecked` context, overflows are not reported and any significant high-order bits outside the range of the result type are discarded.

- Floating-point addition:

```
float operator +(float x, float y);
double operator +(double x, double y);
```

  The sum is computed according to the rules of IEEE 754 arithmetic. The following table lists the results of all possible combinations of nonzero finite values, zeros, infinities, and NaN's. In the table, `x` and `y` are nonzero finite values, and `z` is the result of `x + y`. If `x` and `y` have the same magnitude but opposite signs, `z` is positive zero. If `x + y` is too large to represent in the destination type, `z` is an infinity with the same sign as `x + y`. If `x + y` is too small to represent in the destination type, `z` is a zero with the same sign as `x + y`.

	y	+0	−0	+∞	−∞	NaN
x	z	x	x	+∞	−∞	NaN
+0	y	+0	+0	+∞	−∞	NaN
−0	y	+0	−0	+∞	−∞	NaN
+∞	+∞	+∞	+∞	+∞	NaN	NaN
−∞	−∞	−∞	−∞	NaN	−∞	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Decimal addition:

```
decimal operator +(decimal x, decimal y);
```

If the resulting value is too large to represent in the `decimal` format, a `System.OverflowException` is thrown. The scale of the result, before any rounding, is the larger of the scales of the two operands.

Decimal addition is equivalent to using the addition operator of type `System.Decimal`.

- Enumeration addition. Every enumeration type implicitly provides the following predefined operators, where `E` is the enum type, and `U` is the underlying type of `E`:

```
E operator +(E x, U y);
E operator +(U x, E y);
```

The operators are evaluated exactly as `(E)((U)x + (U)y)`.

- String concatenation:

```
string operator +(string x, string y);
string operator +(string x, object y);
string operator +(object x, string y);
```

The binary + operator performs string concatenation when one or both operands are of type `string`. If an operand of string concatenation is `null`, an empty string is substituted. Otherwise, any non-string argument is converted to its string representation by invoking the virtual `ToString` method inherited from type `object`. If `ToString` returns `null`, an empty string is substituted. [*Example:*

```
using System;
class Test
{
   static void Main() {
      string s = null;
      Console.WriteLine("s = >" + s + "<"); // displays s = ><
      int i = 1;
      Console.WriteLine("i = " + i);        // displays i = 1
      float f = 1.2300E+15F;
      Console.WriteLine("f = " + f);        // displays f = 1.23E+15
      decimal d = 2.900m;
      Console.WriteLine("d = " + d);        // displays d = 2.900
   }
}
```

end example]

The result of the string concatenation operator is a string that consists of the characters of the left operand followed by the characters of the right operand. The string concatenation operator never returns a `null` value. A `System.OutOfMemoryException` may be thrown if there is not enough memory available to allocate the resulting string.

- Delegate combination. Every delegate type implicitly provides the following predefined operator, where `D` is the delegate type:

```
D operator +(D x, D y);
```

The binary + operator performs delegate combination when both operands are of some delegate type D. (If the operands have different delegate types, a compile-time error occurs.) If the first operand is `null`,

the result of the operation is the value of the second operand (even if that is also `null`). Otherwise, if the second operand is `null`, then the result of the operation is the value of the first operand. Otherwise, the result of the operation is a new delegate instance that, when invoked, invokes the first operand and then invokes the second operand. [*Note:* For examples of delegate combination, see §14.7.5 and §22.3. Since `System.Delegate` is not a delegate type, `operator +` is not defined for it. *end note*]

14.7.5 Subtraction operator

For an operation of the form `x - y`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined subtraction operators are listed below. The operators all subtract `y` from `x`.

- Integer subtraction:

```
int operator -(int x, int y);
uint operator -(uint x, uint y);
long operator -(long x, long y);
ulong operator -(ulong x, ulong y);
```

  In a `checked` context, if the difference is outside the range of the result type, a `System.OverflowException` is thrown. In an `unchecked` context, overflows are not reported and any significant high-order bits outside the range of the result type are discarded.

- Floating-point subtraction:

```
float operator -(float x, float y);
double operator -(double x, double y);
```

  The difference is computed according to the rules of IEEE 754 arithmetic. The following table lists the results of all possible combinations of nonzero finite values, zeros, infinities, and NaNs. In the table, `x` and `y` are nonzero finite values, and `z` is the result of `x - y`. If `x` and `y` are equal, `z` is positive zero. If `x - y` is too large to represent in the destination type, `z` is an infinity with the same sign as `x - y`. If `x - y` is too small to represent in the destination type, `z` is a zero with the same sign as `x - y`.

	y	+0	−0	+∞	−∞	NaN
x	z	x	x	−∞	+∞	NaN
+0	−y	+0	+0	−∞	+∞	NaN
−0	−y	−0	+0	−∞	+∞	NaN
+∞	+∞	+∞	+∞	NaN	+∞	NaN
−∞	−∞	−∞	−∞	−∞	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN

- Decimal subtraction:

```
decimal operator -(decimal x, decimal y);
```

  If the resulting value is too large to represent in the `decimal` format, a `System.OverflowException` is thrown. The scale of the result, before any rounding, is the larger of the scales of the two operands.

  Decimal subtraction is equivalent to using the subtraction operator of type `System.Decimal`.

- Enumeration subtraction. Every enumeration type implicitly provides the following predefined operator, where `E` is the enum type, and `U` is the underlying type of `E`:

```
U operator -(E x, E y);
```

  This operator is evaluated exactly as `(U)((U)x - (U)y)`. In other words, the operator computes the difference between the ordinal values of `x` and `y`, and the type of the result is the underlying type of the enumeration.

```
E operator –(E x, U y);
```

This operator is evaluated exactly as `(E)((U)x – y)`. In other words, the operator subtracts a value from the underlying type of the enumeration, yielding a value of the enumeration.

- Delegate removal. Every delegate type implicitly provides the following predefined operator, where `D` is the delegate type:

```
D operator –(D x, D y);
```

The binary `–` operator performs delegate removal when both operands are of some delegate type `D`. (If the operands have different delegate types, a compile-time error occurs.) If the first operand is `null`, the result of the operation is `null`. Otherwise, if the second operand is `null`, then the result of the operation is the value of the first operand. Otherwise, both operands represent invocation lists (§22.1) having one or more entries, and the result is a new invocation list consisting of the first operand's list with the second operand's entries removed from it, provided the second operand's list is a proper contiguous subset of the first's. (For determining subset equality, corresponding entries are compared as for the delegate equality operator (§14.9.8).) Otherwise, the result is the value of the left operand. Neither of the operands' lists is changed in the process. If the second operand's list matches multiple subsets of contiguous entries in the first operand's list, the right-most matching subset of contiguous entries is removed. If removal results in an empty list, the result is `null`. [*Example:* For example:

```
using System;
delegate void D(int x);
class Test
{
   public static void M1(int i) { /* … */ }
   public static void M2(int i) { /* … */ }
}

class Demo
{
   static void Main() {
      D cd1 = new D(Test.M1);
      D cd2 = new D(Test.M2);

      D cd3 = cd1 + cd2 + cd2 + cd1;   // M1 + M2 + M2 + M1
      cd3 -= cd1;                      // => M1 + M2 + M2

      cd3 = cd1 + cd2 + cd2 + cd1;     // M1 + M2 + M2 + M1
      cd3 -= cd1 + cd2;                // => M2 + M1

      cd3 = cd1 + cd2 + cd2 + cd1;     // M1 + M2 + M2 + M1
      cd3 -= cd2 + cd2;                // => M1 + M1

      cd3 = cd1 + cd2 + cd2 + cd1;     // M1 + M2 + M2 + M1
      cd3 -= cd2 + cd1;                // => M1 + M2

      cd3 = cd1 + cd2 + cd2 + cd1;     // M1 + M2 + M2 + M1
      cd3 -= cd1 + cd1;                // => M1 + M2 + M2 + M1
   }
}
```

end example]

14.8 Shift operators

The << and >> operators are used to perform bit shifting operations.

shift-expression:
additive-expression
shift-expression `<<` *additive-expression*
shift-expression `>>` *additive-expression*

For an operation of the form `x << count` or `x >> count`, binary operator overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

When declaring an overloaded shift operator, the type of the first operand must always be the class or struct containing the operator declaration, and the type of the second operand must always be `int`.

The predefined shift operators are listed below.

- Shift left:

```
int operator <<(int x, int count);
uint operator <<(uint x, int count);
long operator <<(long x, int count);
ulong operator <<(ulong x, int count);
```

  The `<<` operator shifts `x` left by a number of bits computed as described below.

  The high-order bits outside the range of the result type of `x` are discarded, the remaining bits are shifted left, and the low-order empty bit positions are set to zero.

- Shift right:

```
int operator >>(int x, int count);
uint operator >>(uint x, int count);
long operator >>(long x, int count);
ulong operator >>(ulong x, int count);
```

  The `>>` operator shifts `x` right by a number of bits computed as described below.

  When `x` is of type `int` or `long`, the low-order bits of `x` are discarded, the remaining bits are shifted right, and the high-order empty bit positions are set to zero if `x` is non-negative and set to one if `x` is negative.

  When `x` is of type `uint` or `ulong`, the low-order bits of `x` are discarded, the remaining bits are shifted right, and the high-order empty bit positions are set to zero.

For the predefined operators, the number of bits to shift is computed as follows:

- When the type of `x` is `int` or `uint`, the shift count is given by the low-order five bits of `count`. In other words, the shift count is computed from `count & 0x1F`.

- When the type of `x` is `long` or `ulong`, the shift count is given by the low-order six bits of `count`. In other words, the shift count is computed from `count & 0x3F`.

If the resulting shift count is zero, the shift operators simply return the value of `x`.

Shift operations never cause overflows and produce the same results in `checked` and `unchecked` contexts.

When the left operand of the `>>` operator is of a signed integral type, the operator performs an *arithmetic* shift right wherein the value of the most significant bit (the sign bit) of the operand is propagated to the high-order empty bit positions. When the left operand of the `>>` operator is of an unsigned integral type, the operator performs a *logical* shift right wherein high-order empty bit positions are always set to zero. To perform the opposite operation of that inferred from the operand type, explicit casts can be used. For example, if `x` is a variable of type `int`, the operation `unchecked((int)((uint)x >> y))` performs a logical shift right of `x`.

14.9 Relational and type-testing operators

The `==`, `!=`, `<`, `>`, `<=`, `>=`, `is` and `as` operators are called the relational and type-testing operators.

relational-expression:
    shift-expression
    relational-expression `<` *shift-expression*
    relational-expression `>` *shift-expression*
    relational-expression `<=` *shift-expression*
    relational-expression `>=` *shift-expression*
    relational-expression `is` *type*
    relational-expression `as` *type*

equality-expression:
relational-expression
equality-expression `==` *relational-expression*
equality-expression `!=` *relational-expression*

The `is` operator is described in §14.9.9 and the `as` operator is described in §14.9.10.

The `==`, `!=`, `<`, `>`, `<=` and `>=` operators are ***comparison operators***. For an operation of the form `x` *op* `y`, where *op* is a comparison operator, overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined comparison operators are described in the following sections. All predefined comparison operators return a result of type `bool`, as described in the following table.

Operation	Result
`x == y`	`true` if `x` is equal to `y`, `false` otherwise
`x != y`	`true` if `x` is not equal to `y`, `false` otherwise
`x < y`	`true` if `x` is less than `y`, `false` otherwise
`x > y`	`true` if `x` is greater than `y`, `false` otherwise
`x <= y`	`true` if `x` is less than or equal to `y`, `false` otherwise
`x >= y`	`true` if `x` is greater than or equal to `y`, `false` otherwise

14.9.1 Integer comparison operators

The predefined integer comparison operators are:

```
bool operator ==(int x, int y);
bool operator ==(uint x, uint y);
bool operator ==(long x, long y);
bool operator ==(ulong x, ulong y);

bool operator !=(int x, int y);
bool operator !=(uint x, uint y);
bool operator !=(long x, long y);
bool operator !=(ulong x, ulong y);

bool operator <(int x, int y);
bool operator <(uint x, uint y);
bool operator <(long x, long y);
bool operator <(ulong x, ulong y);

bool operator >(int x, int y);
bool operator >(uint x, uint y);
bool operator >(long x, long y);
bool operator >(ulong x, ulong y);

bool operator <=(int x, int y);
bool operator <=(uint x, uint y);
bool operator <=(long x, long y);
bool operator <=(ulong x, ulong y);

bool operator >=(int x, int y);
bool operator >=(uint x, uint y);
bool operator >=(long x, long y);
bool operator >=(ulong x, ulong y);
```

Each of these operators compares the numeric values of the two integer operands and returns a `bool` value that indicates whether the particular relation is `true` or `false`.

14.9.2 Floating-point comparison operators

The predefined floating-point comparison operators are:

```
bool operator ==(float x, float y);
bool operator ==(double x, double y);

bool operator !=(float x, float y);
bool operator !=(double x, double y);

bool operator <(float x, float y);
bool operator <(double x, double y);

bool operator >(float x, float y);
bool operator >(double x, double y);

bool operator <=(float x, float y);
bool operator <=(double x, double y);

bool operator >=(float x, float y);
bool operator >=(double x, double y);
```

The operators compare the operands according to the rules of the IEEE 754 standard:

- If either operand is NaN, the result is `false` for all operators except `!=`, for which the result is `true`. For any two operands, `x != y` always produces the same result as `!(x == y)`. However, when one or both operands are NaN, the <, >, <=, and >= operators *do not* produce the same results as the logical negation of the opposite operator. [*Example:* For example, if either of `x` and `y` is NaN, then `x < y` is `false`, but `!(x >= y)` is `true`. *end example*]

- When neither operand is NaN, the operators compare the values of the two floating-point operands with respect to the ordering

  ```
  −∞ < −max < … < −min < −0.0 == +0.0 < +min < … < +max < +∞
  ```

  where `min` and `max` are the smallest and largest positive finite values that can be represented in the given floating-point format. Notable effects of this ordering are:

  - Negative and positive zeros are considered equal.
  - A negative infinity is considered less than all other values, but equal to another negative infinity.
  - A positive infinity is considered greater than all other values, but equal to another positive infinity.

14.9.3 Decimal comparison operators

The predefined decimal comparison operators are:

```
bool operator ==(decimal x, decimal y);
bool operator !=(decimal x, decimal y);
bool operator <(decimal x, decimal y);

bool operator >(decimal x, decimal y);
bool operator <=(decimal x, decimal y);
bool operator >=(decimal x, decimal y);
```

Each of these operators compares the numeric values of the two decimal operands and returns a `bool` value that indicates whether the particular relation is `true` or `false`. Each decimal comparison is equivalent to using the corresponding relational or equality operator of type `System.Decimal`.

14.9.4 Boolean equality operators

The predefined boolean equality operators are:

```
bool operator ==(bool x, bool y);
bool operator !=(bool x, bool y);
```

The result of == is `true` if both `x` and `y` are `true` or if both `x` and `y` are `false`. Otherwise, the result is `false`.

The result of `!=` is `false` if both `x` and `y` are `true` or if both `x` and `y` are `false`. Otherwise, the result is `true`. When the operands are of type `bool`, the `!=` operator produces the same result as the `^` operator.

14.9.5 Enumeration comparison operators

Every enumeration type implicitly provides the following predefined comparison operators:

```
bool operator ==(E x, E y);
bool operator !=(E x, E y);
bool operator <(E x, E y);
bool operator >(E x, E y);
bool operator <=(E x, E y);
bool operator >=(E x, E y);
```

The result of evaluating `x` *op* `y`, where `x` and `y` are expressions of an enumeration type `E` with an underlying type `U`, and *op* is one of the comparison operators, is exactly the same as evaluating `((U)x)` *op* `((U)y)`. In other words, the enumeration type comparison operators simply compare the underlying integral values of the two operands.

14.9.6 Reference type equality operators

The predefined reference type equality operators are:

```
bool operator ==(object x, object y);
bool operator !=(object x, object y);
```

The operators return the result of comparing the two references for equality or non-equality.

Since the predefined reference type equality operators accept operands of type `object`, they apply to all types that do not declare applicable `operator ==` and `operator !=` members. Conversely, any applicable user-defined equality operators effectively hide the predefined reference type equality operators.

The predefined reference type equality operators require the operands to be *reference-type* values or the value `null`; furthermore, they require that a standard implicit conversion (§13.3.1) exists from the type of either operand to the type of the other operand. Unless both of these conditions are true, a compile-time error occurs. [*Note:* Notable implications of these rules are:

- It is a compile-time error to use the predefined reference type equality operators to compare two references that are known to be different at compile-time. For example, if the compile-time types of the operands are two class types `A` and `B`, and if neither `A` nor `B` derives from the other, then it would be impossible for the two operands to reference the same object. Thus, the operation is considered a compile-time error.

- The predefined reference type equality operators do not permit value type operands to be compared. Therefore, unless a struct type declares its own equality operators, it is not possible to compare values of that struct type.

- The predefined reference type equality operators never cause boxing operations to occur for their operands. It would be meaningless to perform such boxing operations, since references to the newly allocated boxed instances would necessarily differ from all other references.

end note]

For an operation of the form `x == y` or `x != y`, if any applicable `operator ==` or `operator !=` exists, the operator overload resolution (§14.2.4) rules will select that operator instead of the predefined reference type equality operator. However, it is always possible to select the predefined reference type equality operator by explicitly casting one or both of the operands to type `object`. [*Example:* The example

```
Using System;
class Test
{
   static void Main() {
      string s = "Test";
      string t = string.Copy(s);
      Console.WriteLine(s == t);
      Console.WriteLine((object)s == t);
      Console.WriteLine(s == (object)t);
      Console.WriteLine((object)s == (object)t);
   }
}
```

produces the output

```
True
False
False
False
```

The `s` and `t` variables refer to two distinct `string` instances containing the same characters. The first comparison outputs `True` because the predefined string equality operator (§14.9.7) is selected when both operands are of type `string`. The remaining comparisons all output `False` because the predefined reference type equality operator is selected when one or both of the operands are of type `object`.

Note that the above technique is not meaningful for value types. The example

```
class Test
{
   static void Main() {
      int i = 123;
      int j = 123;
      System.Console.WriteLine((object)i == (object)j);
   }
}
```

outputs `False` because the casts create references to two separate instances of boxed `int` values. *end example*]

14.9.7 String equality operators

The predefined string equality operators are: :

```
bool operator ==(string x, string y);
bool operator !=(string x, string y);
```

Two `string` values are considered equal when one of the following is true:

- Both values are `null`.
- Both values are non-null references to string instances that have identical lengths and identical characters in each character position.

The string equality operators compare string *values* rather than string *references*. When two separate string instances contain the exact same sequence of characters, the values of the strings are equal, but the references are different. [*Note:* As described in §14.9.6, the reference type equality operators can be used to compare string references instead of string values. *end note*]

14.9.8 Delegate equality operators

Every delegate type implicitly provides the following predefined comparison operators: :

```
bool operator ==(System.Delegate x, System.Delegate y);
bool operator !=(System.Delegate x, System.Delegate y);
```

Two delegate instances are considered equal as follows:

- If either of the delegate instances is `null`, they are equal if and only if both are `null`.
- If either of the delegate instances has an invocation list (§22.1) containing one entry, they are equal if and only if the other also has an invocation list containing one entry, and either:
  - Both refer to the same static method, or
  - Both refer to the same non-static method on the same target object.
- If either of the delegate instances has an invocation list containing two or more entries, those instances are equal if and only if their invocation lists are the same length, and each entry in one's invocation list is equal to the corresponding entry, in order, in the other's invocation list.

Note that delegates of different types can be considered equal by the above definition, as long as they have the same return type and parameter types.

14.9.9 The `is` operator

The `is` operator is used to dynamically check if the run-time type of an object is compatible with a given type. The result of the operation `e is T`, where `e` is an expression and `T` is a type, is a boolean value indicating whether `e` can successfully be converted to type `T` by a reference conversion, a boxing conversion, or an unboxing conversion. The operation is evaluated as follows:

- If the compile-time type of `e` is the same as `T`, or if an implicit reference conversion (§13.1.4) or boxing conversion (§13.1.5) exists from the compile-time type of `e` to `T`:
  - If `e` is of a reference type, the result of the operation is equivalent to evaluating `e != null`.
  - If `e` is of a value type, the result of the operation is `true`.
- Otherwise, if an explicit reference conversion (§13.2.3) or unboxing conversion (§13.2.4) exists from the compile-time type of `e` to `T`, a dynamic type check is performed:
  - If the value of `e` is `null`, the result is `false`.
  - Otherwise, let `R` be the run-time type of the instance referenced by `e`. If `R` and `T` are the same type, if `R` is a reference type and an implicit reference conversion from `R` to `T` exists, or if `R` is a value type and `T` is an interface type that is implemented by `R`, the result is `true`.
  - Otherwise, the result is `false`.
- Otherwise, no reference or boxing conversion of `e` to type `T` is possible, and the result of the operation is `false`.

Note that the `is` operator only considers reference conversions, boxing conversions, and unboxing conversions. Other conversions, such as user defined conversions, are not considered by the `is` operator.

14.9.10 The `as` operator

The `as` operator is used to explicitly convert a value to a given reference type using a reference conversion or a boxing conversion. Unlike a cast expression (§14.6.6), the `as` operator never throws an exception. Instead, if the indicated conversion is not possible, the resulting value is `null`.

In an operation of the form `e as T`, `e` must be an expression and `T` must be a reference type. The type of the result is `T`, and the result is always classified as a value. The operation is evaluated as follows:

- If the compile-time type of `e` is the same as `T`, the result is simply the value of `e`.
- Otherwise, if an implicit reference conversion (§13.1.4) or boxing conversion (§13.1.5) exists from the compile-time type of e to `T`, this conversion is performed and becomes the result of the operation.
- Otherwise, if an explicit reference conversion (§13.2.3) exists from the compile-time type of `e` to `T`, a dynamic type check is performed:
  - If the value of e is `null`, the result is the value `null` with the compile-time type `T`.

- Otherwise, let R be the run-time type of the instance referenced by e. If R and T are the same type, if R is a reference type and an implicit reference conversion from R to T exists, or if R is a value type and T is an interface type that is implemented by R, the result is the reference given by e with the compile-time type T.
- Otherwise, the result is the value null with the compile-time type T.

- Otherwise, the indicated conversion is never possible, and a compile-time error occurs.

Note that the as operator only performs reference conversions and boxing conversions. Other conversions, such as user defined conversions, are not possible with the as operator and should instead be performed using cast expressions.

14.10 Logical operators

The &, ^, and | operators are called the logical operators.

and-expression:
equality-expression
and-expression & *equality-expression*

exclusive-or-expression:
and-expression
exclusive-or-expression ^ *and-expression*

inclusive-or-expression:
exclusive-or-expression
inclusive-or-expression | *exclusive-or-expression*

For an operation of the form x *op* y, where *op* is one of the logical operators, overload resolution (§14.2.4) is applied to select a specific operator implementation. The operands are converted to the parameter types of the selected operator, and the type of the result is the return type of the operator.

The predefined logical operators are described in the following sections.

14.10.1 Integer logical operators

The predefined integer logical operators are:

```
int operator &(int x, int y);
uint operator &(uint x, uint y);
long operator &(long x, long y);
ulong operator &(ulong x, ulong y);

int operator |(int x, int y);
uint operator |(uint x, uint y);
long operator |(long x, long y);
ulong operator |(ulong x, ulong y);

int operator ^(int x, int y);
uint operator ^(uint x, uint y);
long operator ^(long x, long y);
ulong operator ^(ulong x, ulong y);
```

The & operator computes the bitwise logical AND of the two operands, the | operator computes the bitwise logical OR of the two operands, and the ^ operator computes the bitwise logical exclusive OR of the two operands. No overflows are possible from these operations.

14.10.2 Enumeration logical operators

Every enumeration type E implicitly provides the following predefined logical operators:

```
E operator &(E x, E y);
E operator |(E x, E y);
E operator ^(E x, E y);
```

The result of evaluating `x` *op* `y`, where `x` and `y` are expressions of an enumeration type `E` with an underlying type `U`, and *op* is one of the logical operators, is exactly the same as evaluating `(E)((U)x` *op* `(U)y)`. In other words, the enumeration type logical operators simply perform the logical operation on the underlying type of the two operands.

14.10.3 Boolean logical operators

The predefined boolean logical operators are:

```
bool operator &(bool x, bool y);
bool operator |(bool x, bool y);
bool operator ^(bool x, bool y);
```

The result of `x & y` is `true` if both `x` and `y` are `true`. Otherwise, the result is `false`.

The result of `x | y` is `true` if either `x` or `y` is `true`. Otherwise, the result is `false`.

The result of `x ^ y` is `true` if `x` is `true` and `y` is `false`, or `x` is `false` and `y` is `true`. Otherwise, the result is `false`. When the operands are of type `bool`, the `^` operator computes the same result as the `!=` operator.

14.11 Conditional logical operators

The `&&` and `||` operators are called the conditional logical operators. They are also called the "short-circuiting" logical operators.

conditional-and-expression:
inclusive-or-expression
conditional-and-expression `&&` *inclusive-or-expression*

conditional-or-expression:
conditional-and-expression
conditional-or-expression `||` *conditional-and-expression*

The `&&` and `||` operators are conditional versions of the `&` and `|` operators:

- The operation `x && y` corresponds to the operation `x & y`, except that `y` is evaluated only if `x` is `true`.
- The operation `x || y` corresponds to the operation `x | y`, except that `y` is evaluated only if `x` is `false`.

An operation of the form `x && y` or `x || y` is processed by applying overload resolution (§14.2.4) as if the operation was written `x & y` or `x | y`. Then,

- If overload resolution fails to find a single best operator, or if overload resolution selects one of the predefined integer logical operators, a compile-time error occurs.
- Otherwise, if the selected operator is one of the predefined boolean logical operators (§14.10.2), the operation is processed as described in §14.11.1.
- Otherwise, the selected operator is a user-defined operator, and the operation is processed as described in §14.11.2.

It is not possible to directly overload the conditional logical operators. However, because the conditional logical operators are evaluated in terms of the regular logical operators, overloads of the regular logical operators are, with certain restrictions, also considered overloads of the conditional logical operators. This is described further in §14.11.2.

14.11.1 Boolean conditional logical operators

When the operands of `&&` or `||` are of type `bool`, or when the operands are of types that do not define an applicable `operator &` or `operator |`, but do define implicit conversions to `bool`, the operation is processed as follows:

- The operation `x && y` is evaluated as `x ? y : false`. In other words, `x` is first evaluated and converted to type `bool`. Then, if `x` is `true`, `y` is evaluated and converted to type `bool`, and this becomes the result of the operation. Otherwise, the result of the operation is `false`.

- The operation `x || y` is evaluated as `x ? true : y`. In other words, `x` is first evaluated and converted to type `bool`. Then, if `x` is `true`, the result of the operation is `true`. Otherwise, `y` is evaluated and converted to type `bool`, and this becomes the result of the operation.

14.11.2 User-defined conditional logical operators

When the operands of && or || are of types that declare an applicable user-defined `operator &` or `operator |`, both of the following must be true, where `T` is the type in which the selected operator is declared:

- The return type and the type of each parameter of the selected operator must be `T`. In other words, the operator must compute the logical AND or the logical OR of two operands of type `T`, and must return a result of type `T`.

- `T` must contain declarations of `operator true` and `operator false`.

A compile-time error occurs if either of these requirements is not satisfied. Otherwise, the && or || operation is evaluated by combining the user-defined `operator true` or `operator false` with the selected user-defined operator:

- The operation `x && y` is evaluated as `T.false(x) ? x : T.&(x, y)`, where `T.false(x)` is an invocation of the `operator false` declared in `T`, and `T.&(x, y)` is an invocation of the selected `operator &`. In other words, `x` is first evaluated and `operator false` is invoked on the result to determine if `x` is definitely false. Then, if `x` is definitely false, the result of the operation is the value previously computed for `x`. Otherwise, `y` is evaluated, and the selected `operator &` is invoked on the value previously computed for `x` and the value computed for `y` to produce the result of the operation.

- The operation `x || y` is evaluated as `T.true(x) ? x : T.|(x, y)`, where `T.true(x)` is an invocation of the `operator true` declared in `T`, and `T.|(x, y)` is an invocation of the selected `operator |`. In other words, `x` is first evaluated and `operator true` is invoked on the result to determine if `x` is definitely true. Then, if `x` is definitely true, the result of the operation is the value previously computed for `x`. Otherwise, `y` is evaluated, and the selected `operator |` is invoked on the value previously computed for `x` and the value computed for `y` to produce the result of the operation.

In either of these operations, the expression given by `x` is only evaluated once, and the expression given by `y` is either not evaluated or evaluated exactly once.

For an example of a type that implements `operator true` and `operator false`, see §18.4.2.

14.12 Conditional operator

The `?:` operator is called the conditional operator. It is at times also called the ternary operator.

conditional-expression:
    conditional-or-expression
    conditional-or-expression `?` *expression* `:` *expression*

A conditional expression of the form `b ? x : y` first evaluates the condition `b`. Then, if `b` is `true`, `x` is evaluated and becomes the result of the operation. Otherwise, `y` is evaluated and becomes the result of the operation. A conditional expression never evaluates both `x` and `y`.

The conditional operator is right-associative, meaning that operations are grouped from right to left. For example, an expression of the form `a ? b : c ? d : e` is evaluated as `a ? b : (c ? d : e)`.

The first operand of the `?:` operator must be an expression of a type that can be implicitly converted to `bool`, or an expression of a type that implements `operator true`. If neither of these requirements is satisfied, a compile-time error occurs.

The second and third operands of the `?:` operator control the type of the conditional expression. Let `X` and `Y` be the types of the second and third operands. Then,

- If `X` and `Y` are the same type, then this is the type of the conditional expression.
- Otherwise, if an implicit conversion (§13.1) exists from `X` to `Y`, but not from `Y` to `X`, then `Y` is the type of the conditional expression.
- Otherwise, if an implicit conversion (§13.1) exists from `Y` to `X`, but not from `X` to `Y`, then `X` is the type of the conditional expression.
- Otherwise, no expression type can be determined, and a compile-time error occurs.

The run-time processing of a conditional expression of the form `b ? x : y` consists of the following steps:

- First, `b` is evaluated, and the `bool` value of `b` is determined:
  - If an implicit conversion from the type of `b` to `bool` exists, then this implicit conversion is performed to produce a `bool` value.
  - Otherwise, the `operator true` defined by the type of `b` is invoked to produce a `bool` value.
- If the `bool` value produced by the step above is `true`, then `x` is evaluated and converted to the type of the conditional expression, and this becomes the result of the conditional expression.
- Otherwise, `y` is evaluated and converted to the type of the conditional expression, and this becomes the result of the conditional expression.

14.13 Assignment operators

The assignment operators assign a new value to a variable, a property, event, or an indexer element.

assignment:
    unary-expression *assignment-operator* *expression*

assignment-operator: *one of*
    `=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=`

The left operand of an assignment must be an expression classified as a variable, a property access, an indexer access, or an event access.

The `=` operator is called the ***simple assignment operator***. It assigns the value of the right operand to the variable, property, or indexer element given by the left operand. The left operand of the simple assignment operator may not be an event access (except as described in §17.7.1). The simple assignment operator is described in §14.13.1.

The operators formed by prefixing a binary operator with an `=` character are called the ***compound assignment operators***. These operators perform the indicated operation on the two operands, and then assign the resulting value to the variable, property, or indexer element given by the left operand. The compound assignment operators are described in §14.13.2.

The `+=` and `-=` operators with an event access expression as the left operand are called the ***event assignment operators***. No other assignment operator is valid with an event access as the left operand. The event assignment operators are described in §14.13.3.

The assignment operators are right-associative, meaning that operations are grouped from right to left. For example, an expression of the form `a = b = c` is evaluated as `a = (b = c)`.

14.13.1 Simple assignment

The `=` operator is called the simple assignment operator. In a simple assignment, the right operand must be an expression of a type that is implicitly convertible to the type of the left operand. The operation assigns the value of the right operand to the variable, property, or indexer element given by the left operand.

The result of a simple assignment expression is the value assigned to the left operand. The result has the same type as the left operand and is always classified as a value.

If the left operand is a property or indexer access, the property or indexer must have a `set` accessor. If this is not the case, a compile-time error occurs.

The run-time processing of a simple assignment of the form `x = y` consists of the following steps:

- If x is classified as a variable:
  - `x` is evaluated to produce the variable.
  - `y` is evaluated and, if required, converted to the type of `x` through an implicit conversion (§13.1).
  - If the variable given by `x` is an array element of a *reference-type*, a run-time check is performed to ensure that the value computed for `y` is compatible with the array instance of which `x` is an element. The check succeeds if `y` is `null`, or if an implicit reference conversion (§13.1.4) exists from the actual type of the instance referenced by `y` to the actual element type of the array instance containing `x`. Otherwise, a `System.ArrayTypeMismatchException` is thrown.
  - The value resulting from the evaluation and conversion of `y` is stored into the location given by the evaluation of `x`.
- If x is classified as a property or indexer access:
  - The instance expression (if `x` is not `static`) and the argument list (if `x` is an indexer access) associated with `x` are evaluated, and the results are used in the subsequent `set` accessor invocation.
  - `y` is evaluated and, if required, converted to the type of `x` through an implicit conversion (§13.1).
  - The `set` accessor of `x` is invoked with the value computed for `y` as its `value` argument.

[*Note:* The array covariance rules (§19.5) permit a value of an array type `A[]` to be a reference to an instance of an array type `B[]`, provided an implicit reference conversion exists from `B` to A. Because of these rules, assignment to an array element of a *reference-type* requires a run-time check to ensure that the value being assigned is compatible with the array instance. In the example

```
string[] sa = new string[10];
object[] oa = sa;

oa[0] = null;              // Ok
oa[1] = "Hello";           // Ok
oa[2] = new ArrayList();   // ArrayTypeMismatchException
```

the last assignment causes a `System.ArrayTypeMismatchException` to be thrown because an instance of `ArrayList` cannot be stored in an element of a `string[]`. *end note*]

When a property or indexer declared in a *struct-type* is the target of an assignment, the instance expression associated with the property or indexer access must be classified as a variable. If the instance expression is classified as a value, a compile-time error occurs. [*Note:* Because of §14.5.4, the same rule also applies to fields. e*nd note*]

[*Example:* Given the declarations:

```
struct Point
{
   int x, y;

   public Point(int x, int y) {
      this.x = x;
      this.y = y;
   }

   public int X {
      get { return x; }
      set { x = value; }
   }
```

```
        public int Y {
            get { return y; }
            set { y = value; }
        }
    }
    struct Rectangle
    {
        Point a, b;
        public Rectangle(Point a, Point b) {
            this.a = a;
            this.b = b;
        }
        public Point A {
            get { return a; }
            set { a = value; }
        }
        public Point B {
            get { return b; }
            set { b = value; }
        }
    }
```

in the example

```
    Point p = new Point();
    p.X = 100;
    p.Y = 100;
    Rectangle r = new Rectangle();
    r.A = new Point(10, 10);
    r.B = p;
```

the assignments to `p.X`, `p.Y`, `r.A`, and `r.B` are permitted because `p` and `r` are variables. However, in the example

```
    Rectangle r = new Rectangle();
    r.A.X = 10;
    r.A.Y = 10;
    r.B.X = 100;
    r.B.Y = 100;
```

the assignments are all invalid, since `r.A` and `r.B` are not variables. *end example*]

14.13.2 Compound assignment

An operation of the form `x` *op*`=` `y` is processed by applying binary operator overload resolution (§14.2.4) as if the operation was written `x` *op* `y`. Then,

- If the return type of the selected operator is *implicitly* convertible to the type of `x`, the operation is evaluated as `x = x` *op* `y`, except that `x` is evaluated only once.
- Otherwise, if the selected operator is a predefined operator, if the return type of the selected operator is *explicitly* convertible to the type of `x`, and if `y` is *implicitly* convertible to the type of `x`, then the operation is evaluated as `x = (T)(x` *op* `y)`, where `T` is the type of `x`, except that `x` is evaluated only once.
- Otherwise, the compound assignment is invalid, and a compile-time error occurs.

The term "evaluated only once" means that in the evaluation of `x` *op* `y`, the results of any constituent expressions of `x` are temporarily saved and then reused when performing the assignment to `x`. [*Example:* For example, in the assignment `A()[B()] +=` `C()`, where `A` is a method returning `int[]`, and `B` and `C` are methods returning `int`, the methods are invoked only once, in the order `A`, `B`, `C`. *end example*]

When the left operand of a compound assignment is a property access or indexer access, the property or indexer must have both a `get` accessor and a `set` accessor. If this is not the case, a compile-time error occurs.

The second rule above permits `x` *op*`= y` to be evaluated as `x = (T)(x` *op* `y)` in certain contexts. The rule exists such that the predefined operators can be used as compound operators when the left operand is of type `sbyte`, `byte`, `short`, `ushort`, or `char`. Even when both arguments are of one of those types, the predefined operators produce a result of type `int`, as described in §14.2.6.2. Thus, without a cast it would not be possible to assign the result to the left operand.

The intuitive effect of the rule for predefined operators is simply that `x` *op*`= y` is permitted if both of `x` *op* `y` and `x = y` are permitted. [*Example:* In the example

```
byte b = 0;
char ch = '\0';
int i = 0;

b += 1;             // Ok
b += 1000;          // Error, b = 1000 not permitted
b += i;             // Error, b = i not permitted
b += (byte)i;       // Ok

ch += 1;            // Error, ch = 1 not permitted
ch += (char)1;      // Ok
```

the intuitive reason for each error is that a corresponding simple assignment would also have been an error. *end example*]

14.13.3 Event assignment

If the left operand of a `+=` or `-=` operator is classified as an event access, then the expression is evaluated as follows:

- The instance expression, if any, of the event access is evaluated.
- The right operand of the `+=` or `-=` operator is evaluated, and, if required, converted to the type of the left operand through an implicit conversion (§13.1).
- An event accessor of the event is invoked, with argument list consisting of the right operand, after evaluation and, if necessary, conversion. If the operator was `+=`, the `add` accessor is invoked; if the operator was `-=`, the `remove` accessor is invoked.

An event assignment expression does not yield a value. Thus, an event assignment expression is valid only in the context of a *statement-expression* (§15.6).

14.14 Expression

An *expression* is either a *conditional-expression* or an *assignment*.

expression:
    conditional-expression
    assignment

14.15 Constant expressions

A *constant-expression* is an expression that can be fully evaluated at compile-time.

constant-expression:
    expression

The type of a constant expression can be one of the following: `sbyte`, `byte`, `short`, `ushort`, `int`, `uint`, `long`, `ulong`, `char`, `float`, `double`, `decimal`, `bool`, `string`, any enumeration type, or the null type. The following constructs are permitted in constant expressions:

- Literals (including the `null` literal).
- References to `const` members of class and struct types.
- References to members of enumeration types.
- Parenthesized sub-expressions, which are themselves constant expressions.
- Cast expressions, provided the target type is one of the types listed above.
- The predefined +, –, `!`, and ~ unary operators.
- The predefined +, –, *, /, %, <<, >>, &, |, ^, &&, ||, ==, `!=`, <, >, <=, and >= binary operators, provided each operand is of a type listed above.
- The `?:` conditional operator.

Whenever an expression is of one of the types listed above and contains only the constructs listed above, the expression is evaluated at compile-time. This is true even if the expression is a sub-expression of a larger expression that contains non-constant constructs.

The compile-time evaluation of constant expressions uses the same rules as run-time evaluation of non-constant expressions, except that where run-time evaluation would have thrown an exception, compile-time evaluation causes a compile-time error to occur.

Unless a constant expression is explicitly placed in an `unchecked` context, overflows that occur in integral-type arithmetic operations and conversions during the compile-time evaluation of the expression always cause compile-time errors (§14.5.12).

Constant expressions occur in the contexts listed below. In these contexts, a compile-time error occurs if an expression cannot be fully evaluated at compile-time.

- Constant declarations (§17.3).
- Enumeration member declarations (§21.30).
- `case` labels of a `switch` statement (§15.7.2).
- `goto case` statements (§15.9.3).
- Dimension lengths in an array creation expression (§14.5.10.2) that includes an initializer.
- Attributes (§24).

An implicit constant expression conversion (§13.1.6) permits a constant expression of type `int` to be converted to `sbyte`, `byte`, `short`, `ushort`, `uint`, or `ulong`, provided the value of the constant expression is within the range of the destination type.

14.16 Boolean expressions

A *boolean-expression* is an expression that yields a result of type `bool`.

boolean-expression:
expression

The controlling conditional expression of an *if-statement* (§15.7.1), *while-statement* (§15.8.1), *do-statement* (§15.8.2), or *for-statement* (§15.8.3) is a *boolean-expression*. The controlling conditional expression of the `?:` operator (§14.12) follows the same rules as a *boolean-expression*, but for reasons of operator precedence is classified as a *conditional-or-expression*.

A *boolean-expression* is required to be of a type that can be implicitly converted to `bool` or of a type that implements `operator true`. [*Note:* As required by §17.9.1, any type that implements `operator true` must also implement `operator false`. *end note*] If neither requirement is satisfied, a compile-time error occurs.

When a boolean expression is of a type that cannot be implicitly converted to `bool` but does implement `operator true`, then following evaluation of the expression, the `operator true` implementation provided by that type is invoked to produce a `bool` value.

[*Note:* The `DBBool` struct type in §18.4.2 provides an example of a type that implements `operator true` and `operator false`. *end note*]